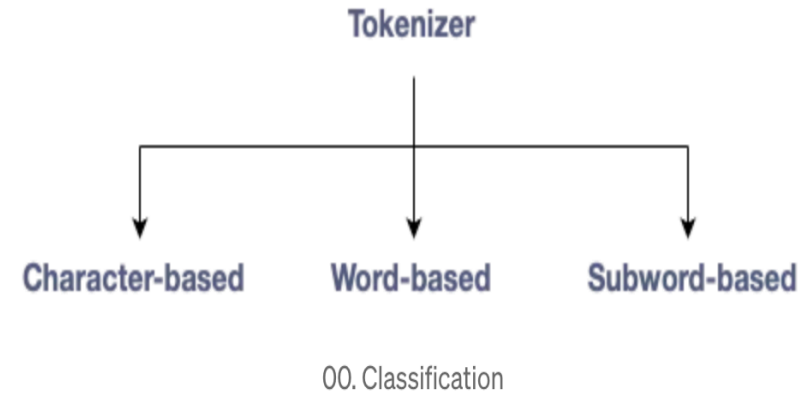
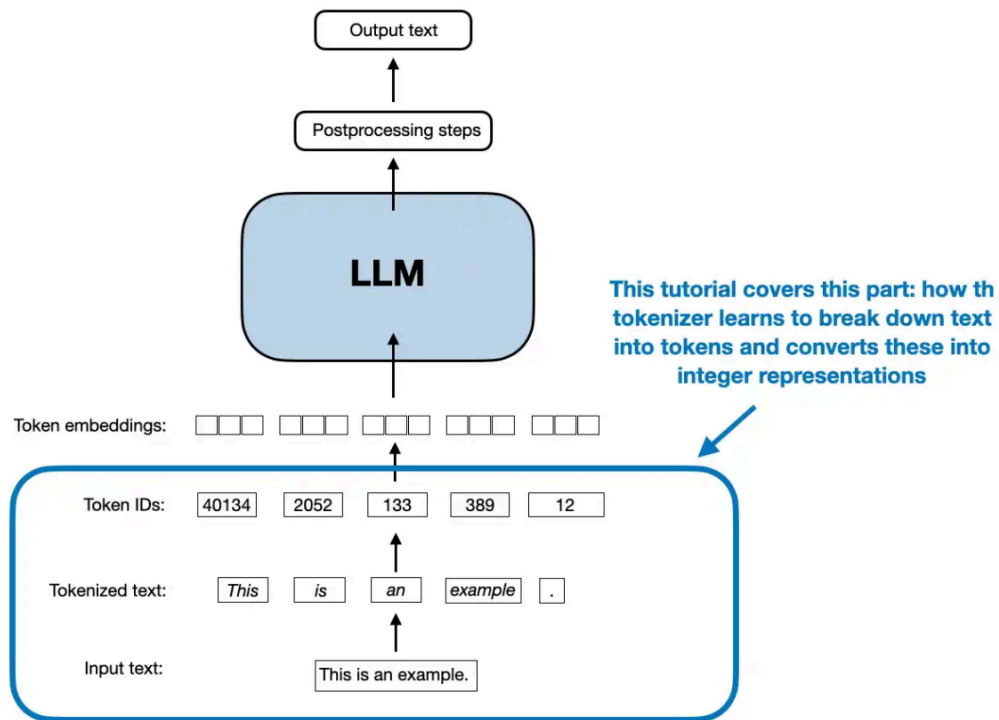


LLms

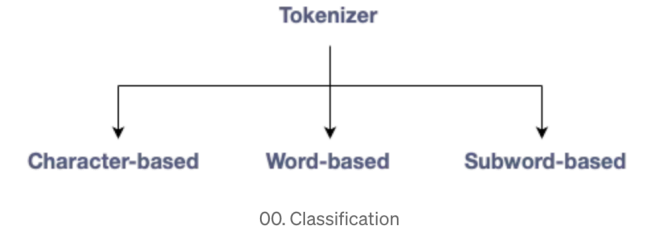
Text representation and tokenization

Tokenizers

What is a tokenizer: A **tokenizer** is the function that turns raw text into a sequence of **tokens** (usually integers) that a language model can process,



Tokenizer



1. Character-Based Tokenizer :

This type of tokenizer split the texts into tokens based on characters.

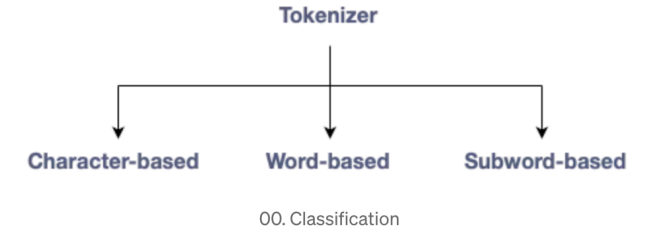
For example, I am studying Tokenizer! would be tokenized into the following tokens :

"I", " ", "a", "m", " ", "s", "t", "y", "d", "y", "i", "n", "g", " ", "T",
"o", "k", "e", "n", "i", "z", "e", "r"

Pro:Reduced Vocabulary Size, full coverage

Cons: too Big sequence, too much information compression

Tokenizer



2. Word-Based Tokenizer :

This type of tokenizer split the text into tokens based on words.

For example, I am studying Tokenizer! would be tokenized into the following

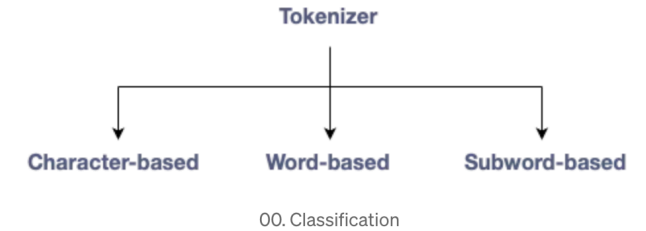
tokens : ["I", "am", "studying", "Tokenizer"]

['I', 'am', 'studying', 'Tokenizer']

Pro: short sequences, allows information expressivity

Cons: requires big vocabulary size, limited coverage

Tokenizer:



BPE (Byte-pair) encoding: Start with text split into tiny pieces (characters/bytes).
Repeatedly find the **most common adjacent pair** of pieces and **merge it into one new piece**

1.3 BPE algorithm outline

1. Identify frequent pairs

- In each iteration, scan the text to find the most commonly occurring pair of bytes (or characters)

2. Replace and record

- Replace that pair with a new placeholder ID (one not already in use, e.g., if we start with 0...255, the first placeholder would be 256)
- Record this mapping in a lookup table
- The size of the lookup table is a hyperparameter, also called "vocabulary size" (for GPT-2, that's 50,257)

3. Repeat until no gains

- Keep repeating steps 1 and 2, continually merging the most frequent pairs
- Stop when no further compression is possible (e.g., no pair occurs more than once)

Decompression (decoding)

- To restore the original text, reverse the process by substituting each ID with its corresponding pair, using the lookup table

Example [\[edit \]](#)

Suppose the data to be encoded is:[\[8\]](#)

```
aaabdaaabc
```

The byte pair "aa" occurs most often, so it will be replaced by a byte that is not used in the data, such as "Z". Now there is the following data and replacement table:

```
ZabdZabc
Z=aa
```

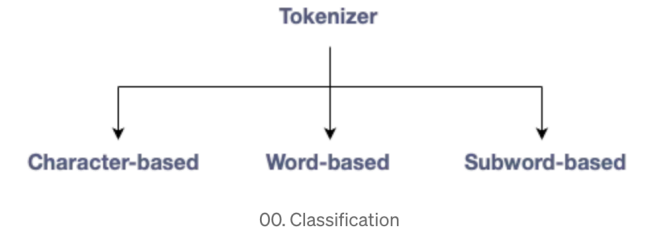
Then the process is repeated with byte pair "ab", replacing it with "Y":

```
ZYdZYac
Y=ab
Z=aa
```

The only literal byte pair left occurs only once, and the encoding might stop here. Alternatively, the process could continue with [recursive](#) byte-pair encoding, replacing "ZY" with "X":

```
XdXac
X=ZY
Y=ab
Z=aa
```

Tokenizer



2. Byte-Level encoding: Same as byte-pair but starting from the representation using UTF-8 encoding. The initial vocabulary is of size 256

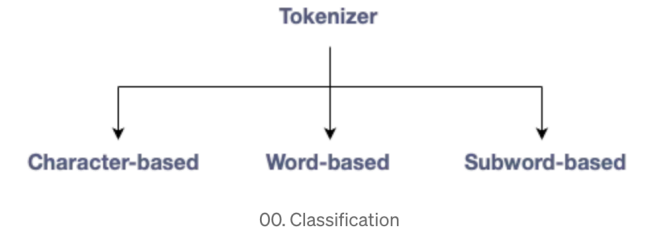
UTF-8 is a standard way to represent *any Unicode text* (letters from **all languages**, emojis, **math symbols**) as a sequence of **bytes**

```
tokens = ("bright kite took flight in the night.").encode("utf-8")
tokens = list(map(int, tokens))

# Output byte-level tokens
# [98, 114, 105, 103, 104, 116, 32, 107, 105, 116, 32, 116, 107, 101, 110, 116, 107, 46]
```

Why preferred over characters? Init voc is only 256 -> full coverage with minimal reduced voc set

Tokenizer



2. Byte-Level encoding: Same as byte-pair but starting from the representation using UTF-8 encoding. The initial vocabulary is of size 256

UTF-8 is a standard way to represent *any Unicode text* (letters from **all languages**, emojis, **math symbols**) as a sequence of **bytes**

```
tokens = ("bright kite took flight in the night.").encode("utf-8")
tokens = list(map(int, tokens))

# Output byte-level tokens
# [98, 114, 105, 103, 104, 116, 32, 107, 105, 116, 101, 32, 116, 111, 111, 107,
```

Why preferred over characters? Init voc is only 256 -> full coverage with minimal reduced voc set

Special tokens

1. **Beginning of Sequence (BOS):** This token indicates the beginning of a sequence. It is often used in tasks such as language modeling or sequence generation to signal the model to start generating text.
 2. **End of Sequence (EOS):** This token marks the end of a sequence. It is used in tasks like language modeling to indicate the completion of text generation.
 3. **Padding Token (PAD):** In machine learning tasks involving sequence data, input sequences often need to have uniform lengths for efficient processing (e.g., in mini-batch training). Padding tokens are inserted to standardize the length of input sequences by filling empty spaces in shorter sequences.
 4. **Unknown Token (UNK):** This token represents words or tokens that are not present in the vocabulary used for tokenization. When encountering out-of-vocabulary words during tokenization, they are replaced with the unknown token to ensure the model can still process them.
 5. **Mask Token (MASK):** This token is used in tasks like masked language modeling, where a portion of the input sequence is masked out, and the model is trained to predict the masked tokens based on the context provided by the surrounding tokens.
-

Sequence packing

Problem: padding wastes GPU compute/memory

Packing: concatenate multiple short samples into one long sequence

Example: S1 <eos> S2 <eos> S3 <eos> (EOS marks boundaries
mask enforces them)

Goal: maximize context window utilization (more real tokens/step)

Benefit: better gpu utilization → faster training

Catch: model must not attend across sample boundaries

->Fix: use segmented / block-diagonal attention masks (plus causal mask)



Sequence packing

Problem: padding wastes GPU compute/memory

Packing: concatenate multiple short samples into one long sequence

Example: S1 <eos> S2 <eos> S3 <eos> (EOS marks boundaries
mask enforces them)

Goal: maximize context window utilization (more real tokens/step)

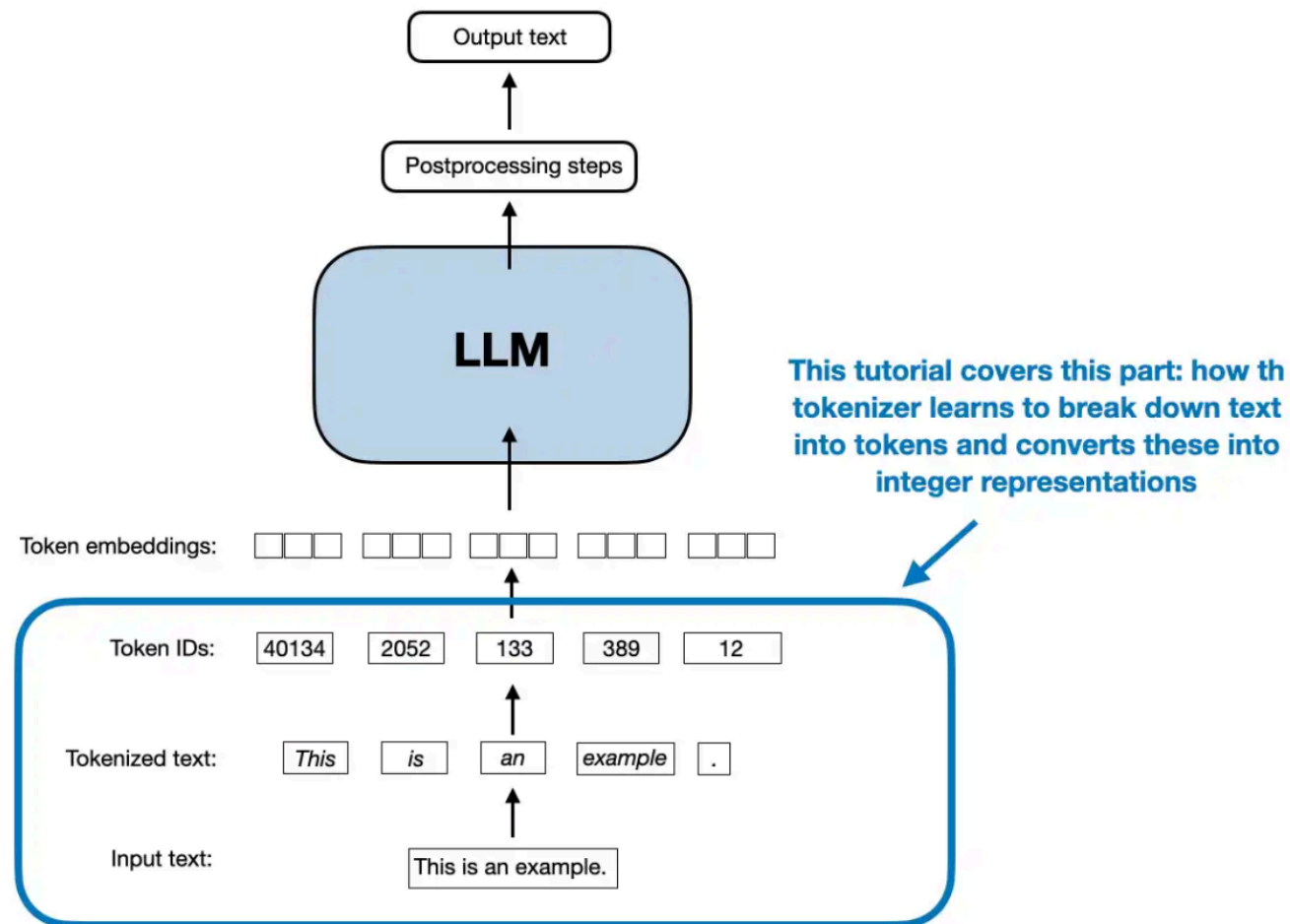
Benefit: better gpu utilization → faster training

Catch: model must not attend across sample boundaries

->Fix: use segmented / block-diagonal attention masks (plus causal mask)



Token embeddings



Token embeddings

- Word (token) embeddings turn discrete tokens into continuous vectors the model can compute with
- They capture semantic + syntactic similarity (similar tokens → similar vectors)
- They're the *front door* of an LLM: everything starts as embeddings
- **How they're used in LLMs**
 - Tokens → **embedding lookup table** (trainable matrix: vocab_size × d_model)
 - Embeddings are combined with **positional encoding** (absolute, relative, rope)
 - Often **weight tying**: input embedding matrix = output (LM head) matrix to reduce params + help generalization

Positional encoding

Same words might have different meaning (different representation) depending on the position on the context

EX: “the **dog** chases the **pig**” and “the pig chases the dog,”

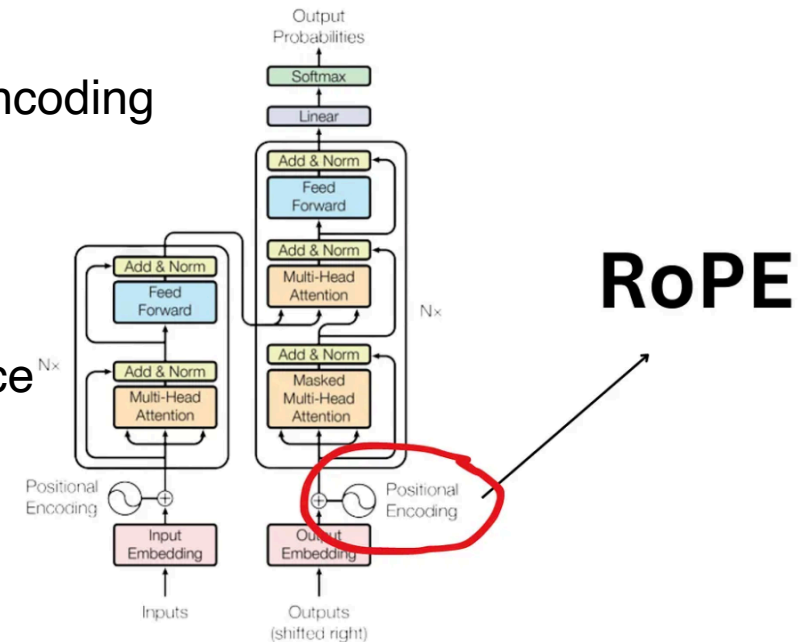
Attention weight pig would be the same for dog without positional encoding because projection matrices are shared across tokens

EX: “the **dog** chases the **pig**” : who chased?

Swapping “dog” and “pic”, would not change the attention importance
“who” it puts on “dog” or “pig” -> permutation equivariance

Why: projections are shared

Solution-> encoder and use positional information



Absolute Positional Embeddings

Learned absolute positional embeddings

learned during training, A learned lookup table p_i added to token embeddings.

pro:

- simple and flexible (look up table)**
- better in-distribution performance** at the trained max length
- **Allows efficient key-value caching**

Cons:

- Limited Sequence Length**
- **Poor relative positional understanding**
- Parameter heavy**

Absolute Positional Embeddings

Sinusoidal positional encoding

It generates a unique vector for each position in the sequence, using a combination of sine and cosine functions.

- The encoding vector has the same dimensionality as the token embeddings, allowing them to be simply added together.

$$h_t^{(0)} = x_t + p_t$$

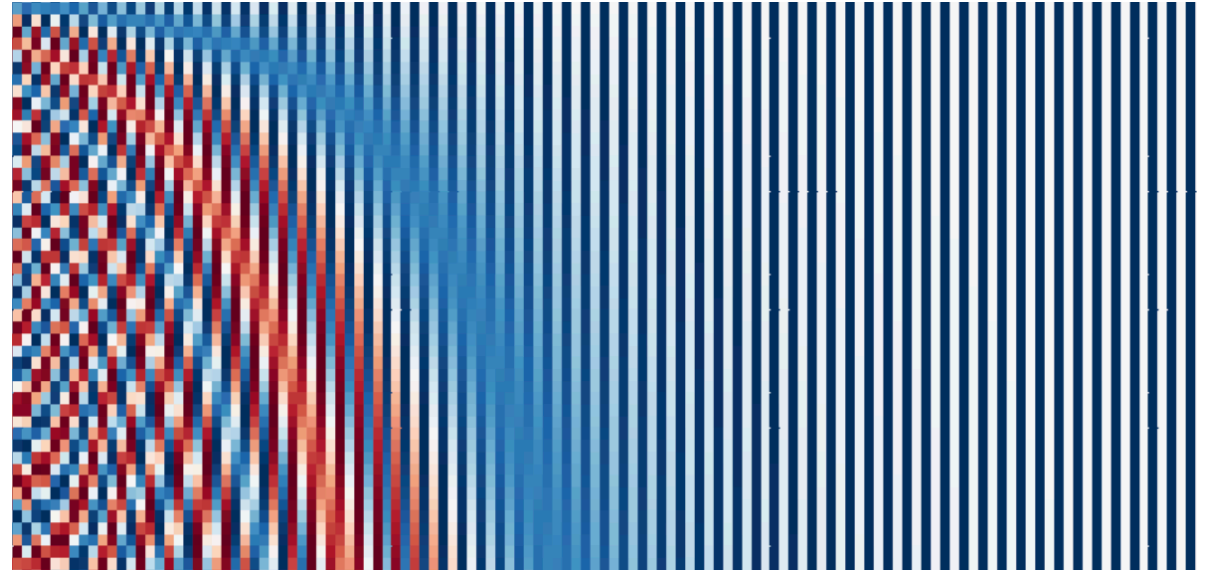
$$p_t[2i] = \sin\left(t/10000^{2i/d}\right), \quad p_t[2i+1] = \cos\left(t/10000^{2i/d}\right)$$

pro:

- **Allows efficient key-value caching**
- **Parameter free**
- **Defined for any length**

Cons:

- **poor generalization above learned max seuquence**
- **Poor relative positional understanding**



Relative Positional Embeddings

- **Relative positional embeddings**
 - Encode **distance** between tokens ($i - j$), not absolute index i
 - Injected by **modifying attention scores**, not by adding a position vector to token embeddings
- **T5 case study (relative position bias)**
 - Learn a **bias value per relative offset bucket** (e.g., "1 apart", "2 apart", ...) shared across positions
 - In self-attention, add this bias to the logits:

$$\text{scores}_{i,j} = \frac{q_i k_j^\top}{\sqrt{d_k}} + b(\text{bucket}(i - j))$$

- So tokens with the same relative distance get the **same bias**, regardless of where they appear

Pros:

- Adapts better to unseen sequence lengths
- Better relative positional understanding

Cons:

- Slower computation
- More complex to optimize (fusing, flash,kv cache)

Rotary Positional Embeddings (RoPE)

- What it is
 - RoPE (Rotary Positional Embedding) encodes position by **rotating** the query and key vectors (not by adding a position vector to token embeddings).

- Where it's applied
 - Applied to **Q and K** *before* attention:

$$q_t = (x_t W_Q), \quad k_t = (x_t W_K)$$

$$\tilde{q}_t = \text{RoPE}(t) q_t, \quad \tilde{k}_t = \text{RoPE}(t) k_t$$

- How the rotation works
 - Split the vector into **2D pairs** (u_{2i}, u_{2i+1}) .
 - For each pair, rotate by angle $\theta_{t,i}$:

$$\begin{pmatrix} u'_{2i} \\ u'_{2i+1} \end{pmatrix} = \begin{pmatrix} \cos \theta_{t,i} & -\sin \theta_{t,i} \\ \sin \theta_{t,i} & \cos \theta_{t,i} \end{pmatrix} \begin{pmatrix} u_{2i} \\ u_{2i+1} \end{pmatrix}$$

- Angles use different frequencies:

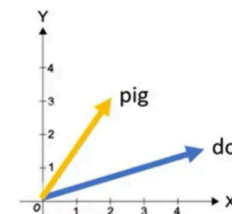
$$\theta_{t,i} = t \cdot \omega_i, \quad \omega_i = 10000^{-2i/d}$$

- Key property
 - The attention dot product becomes sensitive to **relative position**:

$$\tilde{q}_t^\top \tilde{k}_s \text{ depends on } (t - s)$$

- So it behaves like a **relative position method** but implemented via Q/K transforms.

The pig chased the dog



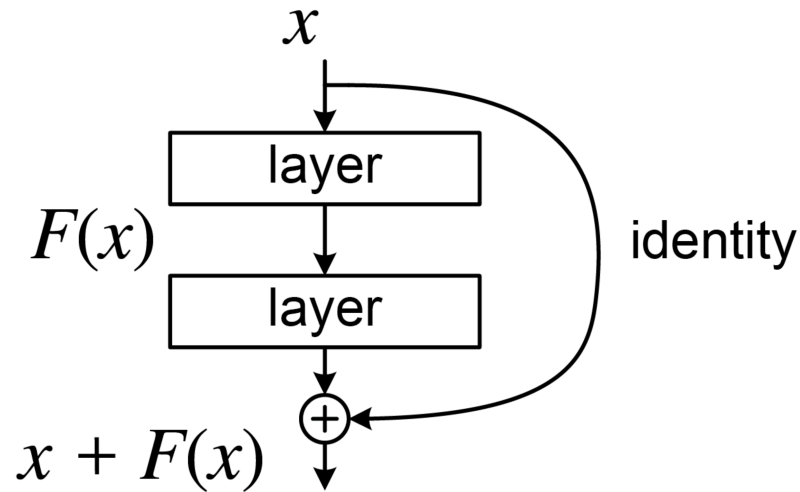
Pros:

- kv cache friendly
- Adapts to any context length
- Captures relative positioning and absolute position
- No attention-bias matrix needed (kernel-friendly / FlashAttention-friendly).

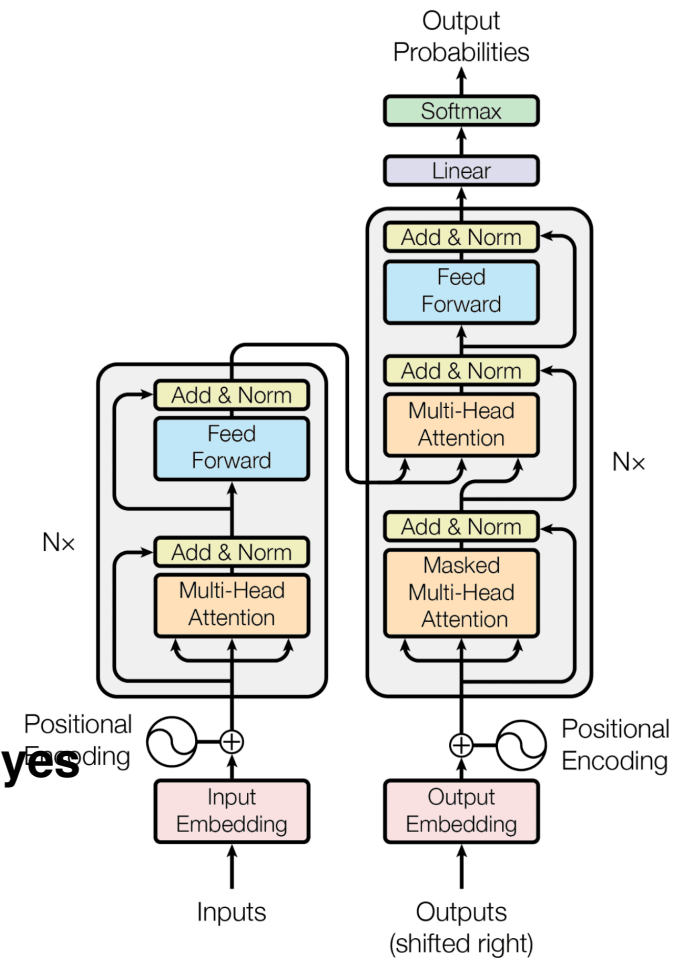
Transformer core architecture

-

Residual (skip) connections



- Solves the Vanishing Gradient Problem
- Allows network to generalize better by skipping non useful layers



LayerNorm (LN) and RMSNorm

- **Goal:** keep activations in a well-behaved scale so training is stable (less exploding/vanishing, smoother optimization).
- ->(normalize across the **feature dimension**, not across time)

LayerNorm (LN)

For a token vector x :

$$\mu = \frac{1}{d} \sum_{i=1}^d x_i, \quad \sigma^2 = \frac{1}{d} \sum_{i=1}^d (x_i - \mu)^2$$

$$\text{LN}(x) = \gamma \odot \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}} + \beta$$

- $\gamma, \beta \in \mathbb{R}^d$ are learnable scale/shift.
- Normalizes **mean and variance**.

RMSNorm

$$\text{rms} = \sqrt{\frac{1}{d} \sum_{i=1}^d x_i^2 + \epsilon}$$

$$\text{RMSNorm}(x) = \gamma \odot \frac{x}{\text{rms}}$$

- Typically **no mean subtraction** (no β in many impls).

Preferred choice: cheaper and faster

Key for attention due to QK computation

Pre-norm vs Post-norm

Post-norm (original Transformer paper style)

$$h_{l+1} = \text{LN}(h_l + F(h_l))$$

Gradient has no direct path to identity ->

Pre-norm (common in modern LLMs)

$$h_{l+1} = h_l + F(\text{LN}(h_l))$$

$$\frac{\partial L}{\partial h_l} = \frac{\partial L}{\partial h_{l+1}} \left(I + \frac{\partial}{\partial h_l} F(\text{LN}(h_l)) \right)$$

Feed-forward network (FFN / MLP)

Purpose: add **nonlinear feature mixing** beyond attention (attention mixes *tokens*, MLP mixes *features*)

Shared weights

Standard FFN (2-layer MLP)

For token hidden vector $h \in \mathbb{R}^{d_{\text{model}}}$:

$$\text{FFN}(h) = W_2 \phi(W_1 h + b_1) + b_2$$

- $W_1 \in \mathbb{R}^{d_{\text{ff}} \times d_{\text{model}}}$
- $W_2 \in \mathbb{R}^{d_{\text{model}} \times d_{\text{ff}}}$
- d_{ff} is the **MLP width**

Dim preserved for skip connection

Feed-forward network (FFN / MLP)

Purpose: add **nonlinear feature mixing** beyond attention (attention mixes *tokens*, MLP mixes *features*)

Shared weights

Standard FFN (2-layer MLP)

For token hidden vector $h \in \mathbb{R}^{d_{\text{model}}}$:

$$\text{FFN}(h) = W_2 \phi(W_1 h + b_1) + b_2$$

- $W_1 \in \mathbb{R}^{d_{\text{ff}} \times d_{\text{model}}}$
- $W_2 \in \mathbb{R}^{d_{\text{model}} \times d_{\text{ff}}}$
- d_{ff} is the **MLP width**

Dim preserved for skip connection

Activations functions (old-style)

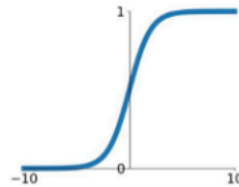
Purpose: the tool we use to add non-linearity in the FNN equations

$$\text{FFN}(h) = W_2 \phi(W_1 h + b_1) + b_2$$

Activation Functions

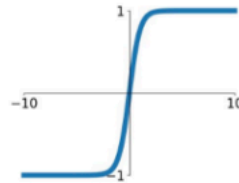
Sigmoid

$$\sigma(x) = \frac{1}{1+e^{-x}}$$



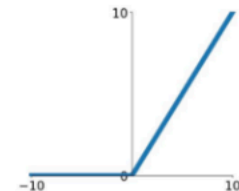
tanh

$$\tanh(x)$$



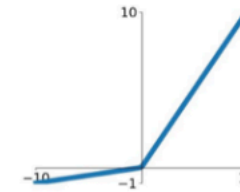
ReLU

$$\max(0, x)$$



Leaky ReLU

$$\max(0.1x, x)$$

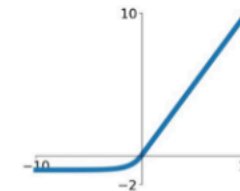


Maxout

$$\max(w_1^T x + b_1, w_2^T x + b_2)$$

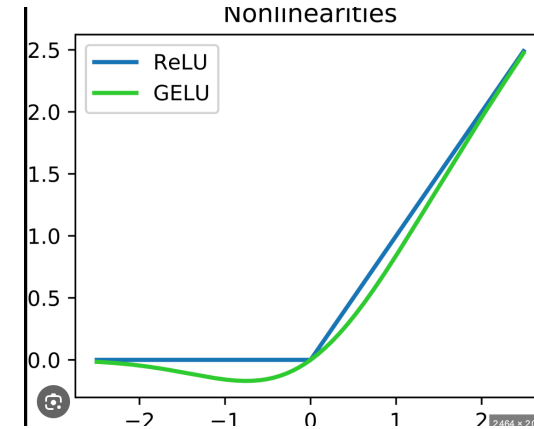
ELU

$$\begin{cases} x & x \geq 0 \\ \alpha(e^x - 1) & x < 0 \end{cases}$$

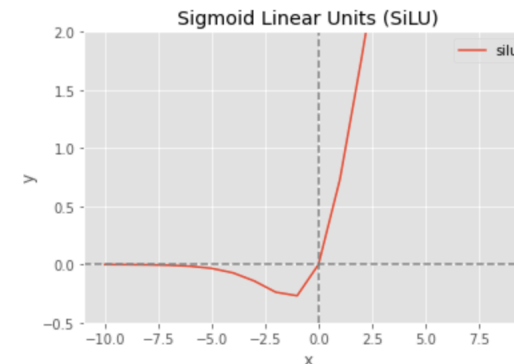


Activations functions (middle-age)

$$\begin{aligned}\text{GELU}(x) &= xP(X \leq x) = x\Phi(x) \\ &\approx 0.5x \left(1 + \tanh \left[\sqrt{2/\pi} (x + 0.044715x^3) \right] \right)\end{aligned}$$



$$\text{SiLU}(x) = x \left(\frac{1}{1 + e^{-x}} \right)$$



gated activation(modern)

SwiGLU (SiLU)

$$\text{FFN}(x) = W_{\text{down}}(\text{silu}(W_{\text{gate}}x) \odot (W_{\text{up}}x))$$

GeGLU (GELU-gated GLU)

$$\text{FFN}(x) = W_{\text{down}}(\text{gelu}(W_{\text{gate}}x) \odot (W_{\text{up}}x))$$

Dropout in Transformers

What dropout does

- Randomly zeroes some activations during training:

$$\tilde{x} = m \odot x, \quad m_i \sim \text{Bernoulli}(1 - p)$$

(then scaled so the expected value stays the same).

- Purpose: reduce co-adaptation → better generalization (less overfitting).

Types: **Embedding dropout**, **Attention dropout**, **Residual / output dropout**, **MLP dropout**, **Layer dropout**

Let $m_l \sim \text{Bernoulli}(1 - p_l)$ be the gate for the residual branch.

A common stochastic-depth form is:

$$h_{l+1} = h_l + \frac{m_l}{1 - p_l} F(h_l)$$

Needed when we want regularization

Not always needed, ex: large data

Attention

-

Attention

What it is: Attention lets each token compute **weights over other tokens** (via query–key similarity) and then form its new representation as a **weighted sum of their values**.

For each token vector $x_t \in \mathbb{R}^{d_{\text{model}}}$, the model makes three projections:

$$q_t = x_t W_Q, \quad k_t = x_t W_K, \quad v_t = x_t W_V$$

- **Query (Q):** "what I'm looking for"
- **Key (K):** "what I contain / what I match on"
- **Value (V):** "the information I will provide if you attend to me"

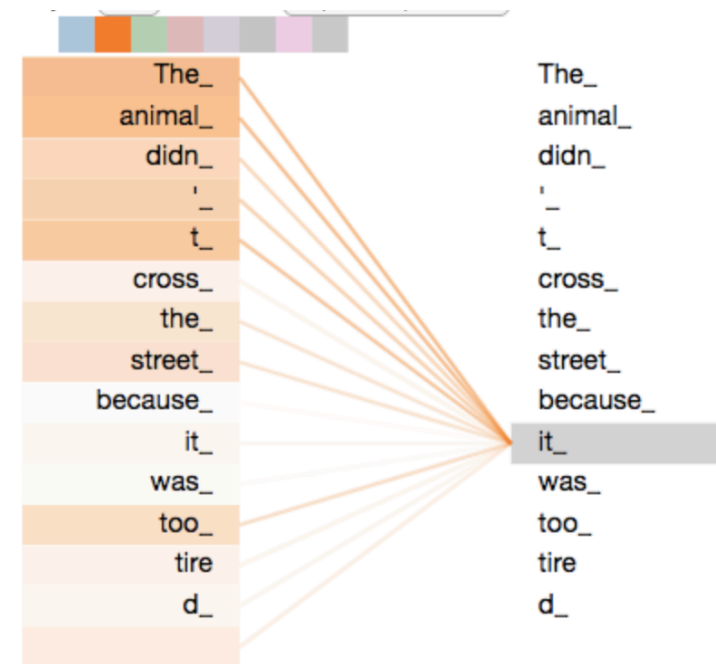
All tokens share the same W_Q, W_K, W_V within a layer.

Pro:

- context-aware token representation
- Highly parallelizable
- Direct long-range communication: any token can attend to any other in a single layer

Cons:

- Quadratic cost



How attention weights are computed

For a token at position i , it scores every token j by similarity:

$$s_{i,j} = \frac{q_i^\top k_j}{\sqrt{d_k}}$$

Scaling factor -> for variance

Optionally add masks/biases:

- **causal mask** (decoder-only) blocks $j > i$
- **padding mask** blocks PAD tokens
- **relative bias** (like T5) adds $b(i - j)$

Then convert scores to probabilities:

$$a_{i,j} = \text{softmax}(s_{i,*})_j$$

So $a_{i,j}$ tells you how much token i attends to token j .

What the output is (the "added value")

The output for token i is a weighted sum of the **values**:

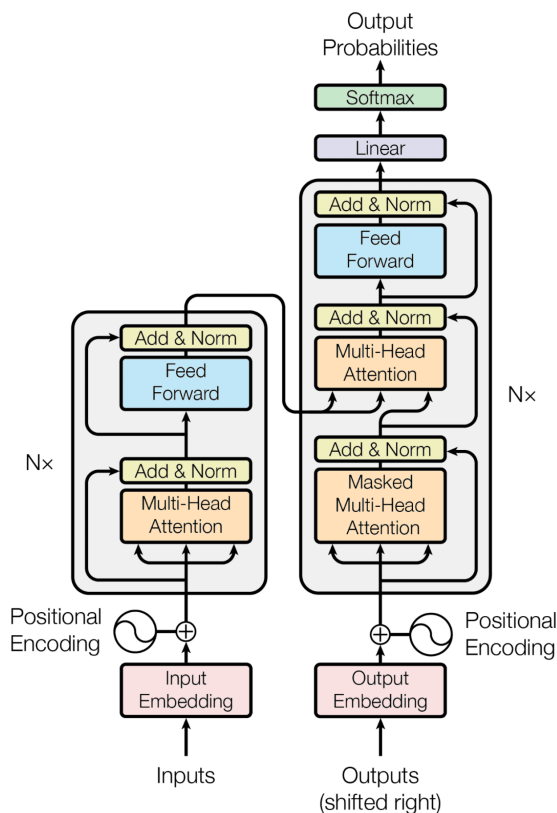
$$o_i = \sum_{j=1}^T a_{i,j} v_j$$

↓

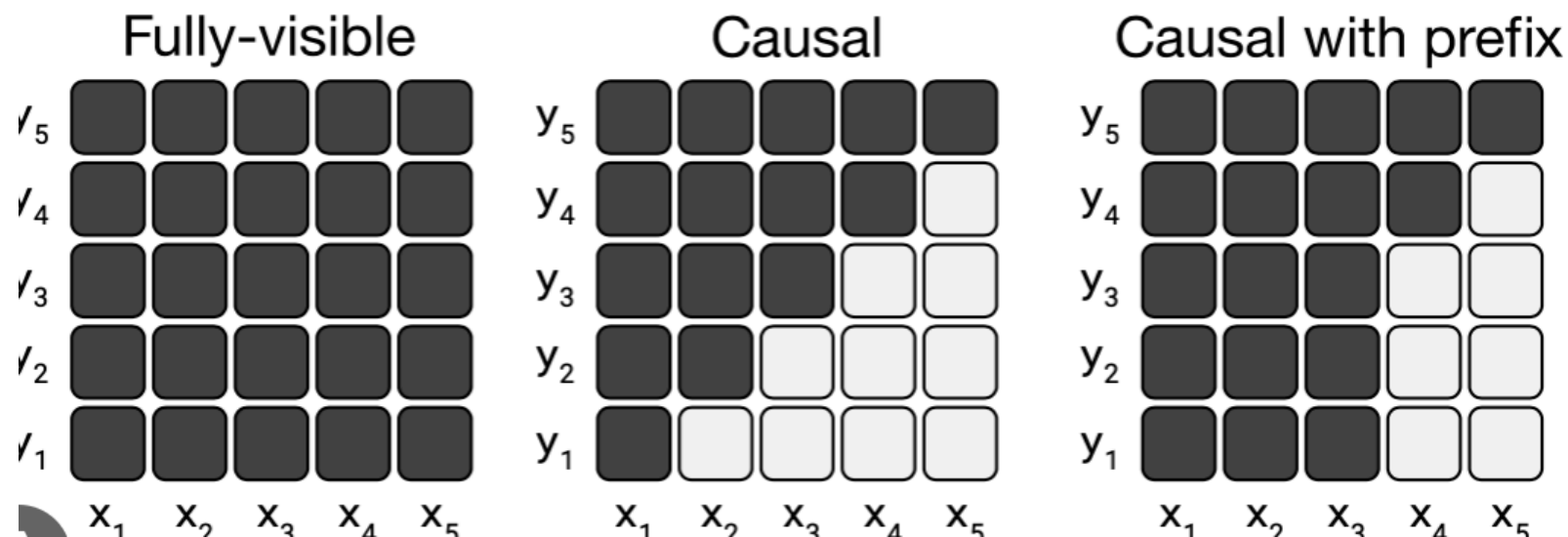
Self-Attention vs Cross-Attention

Self-attention: Self-attention is when the **query, key, and value** all come from the **same input sequence**

Cross-attention: Cross-attention is when the **query** comes from one source, and the **key** and **value** come from another.



Causal masking vs bidirectional masking



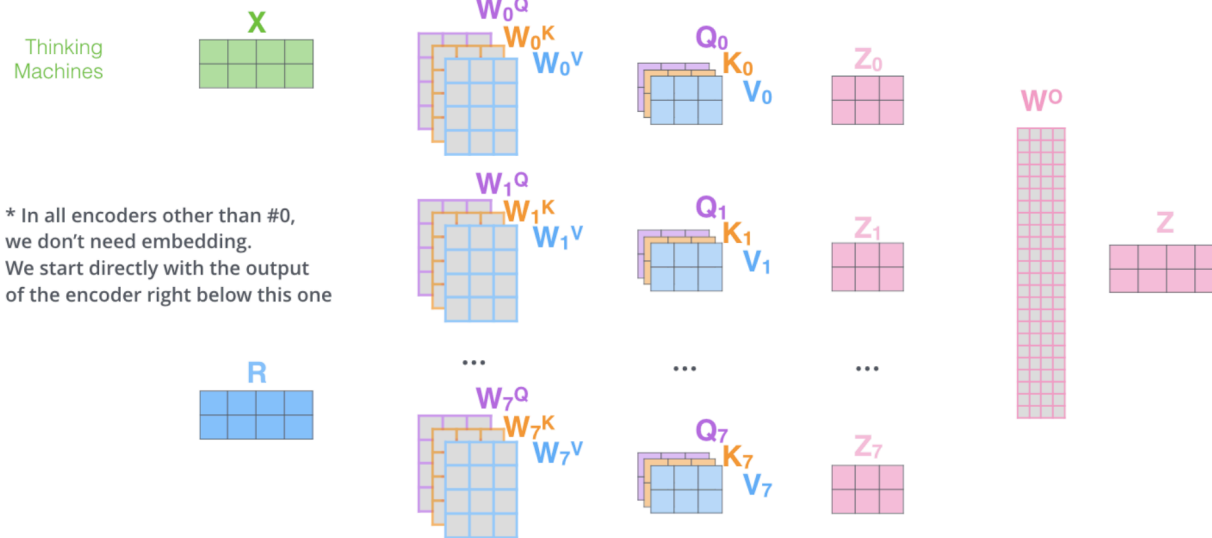
-

$$M_{i,j} = \begin{cases} 0 & j \leq i \\ -\infty & j > i \end{cases}$$

$$M_{i,j} = 0 \text{ if } (i \leq P \wedge j \leq P) \text{ or } (i > P \wedge j \leq i)$$

Multi-head attention

Goal: gives attention layer multiple “representation subspaces”



Shapes

- d_{model} : model width
- h : number of heads
- $d_{\text{head}} = d_k$
- Typically: $d_{\text{model}} = h \cdot d_{\text{head}}$

output projection matrix W allow to mix info across heads

-> Problem KV caching scales $O(t \cdot h \cdot d_{\text{head}})$ during decoding

Multi-query attention

Observation: you often don't need completely different K, V for every head

idea: share **one** K/V across all query heads

- Queries: $Q \in \mathbb{R}^{t \times h_q \times d_{\text{head}}}$
- Keys/Values (shared): $K, V \in \mathbb{R}^{t \times 1 \times d_{\text{head}}}$



KV caching scales $O(t \cdot d_{\text{head}})$ during decoding: GOOD

-

-> Problem poor representation capacity

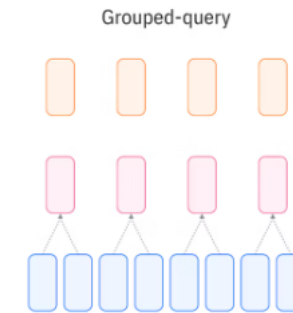
Grouped-query attention

idea: Instead of using only 1 head for keys and values, it uses g heads

$$Q \in \mathbb{R}^{t \times h_q \times d_{\text{head}}}$$

$$K, V \in \mathbb{R}^{t \times h_{kv} \times d_{\text{head}}}$$

$$g = \frac{h_q}{h_{kv}}$$



KV caching scales $O(t \cdot d_{\text{head}} \cdot h/g)$ during decoding:

-

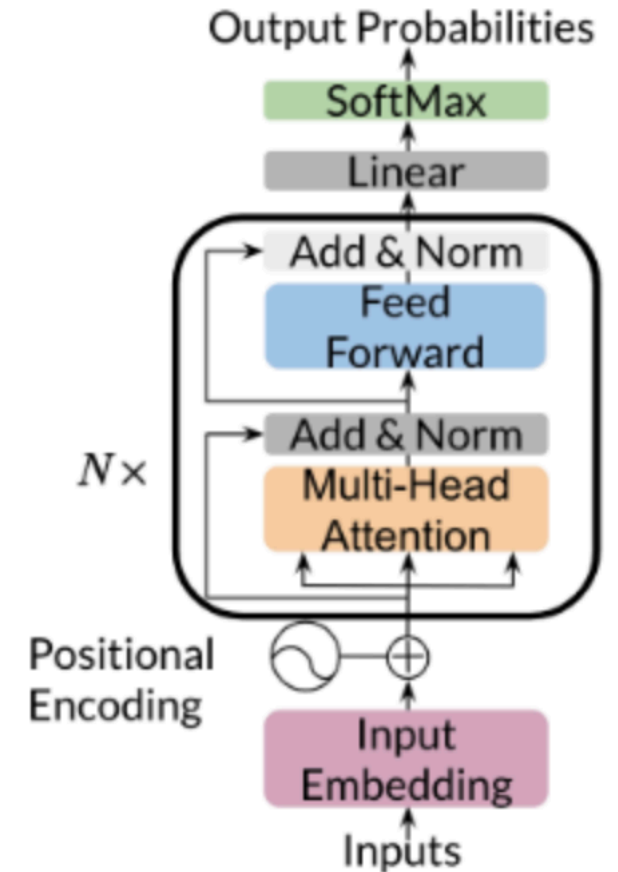
-> Still relatively fast without too much degradation

Model families

Decoder only

- **What:** causal self-attention (can only look left)
- **Used for:** text generation, chat, code, completion (prompt → continuation)
- **Type:** **generative** (autoregressive $p(x) = \prod_t p(x_t | x_{<t})$)
- **Examples:** GPT family, LLaMA, Mistral, Claude-style decoder models

Often preferred for data modeling simplicity and output flexibility



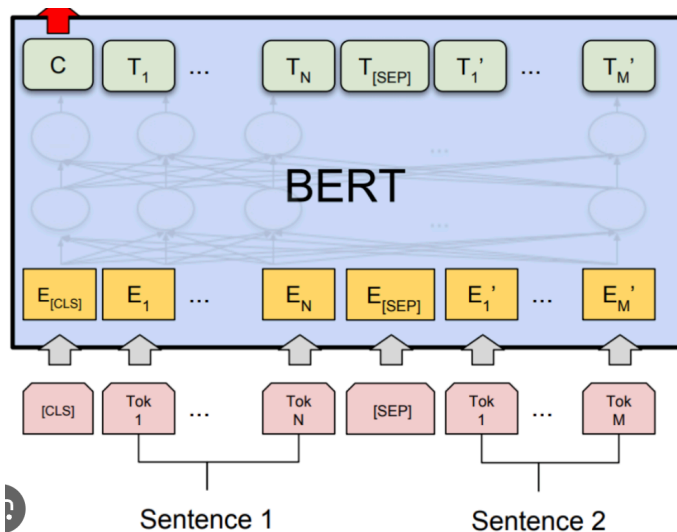
Encoder only

What: full/bidirectional self-attention (looks left + right)

Used for: understanding tasks: classification, retrieval embeddings, NER, sentence similarity

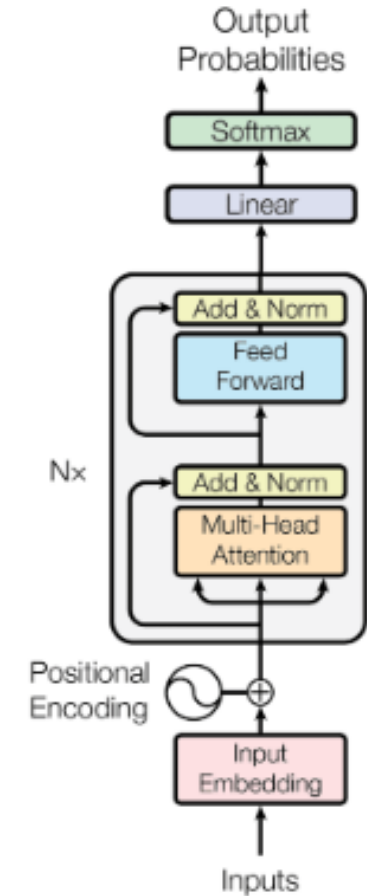
Type: **discriminative / representation** (typically models $p(y | x)$ via a head)

Examples: BERT, RoBERTa, DeBERTa, ELECTRA



Trained with

- *Masked Language Modeling*
- *Next Sentence Prediction*



Encoder-decoder

Encoder-decoder (seq2seq)

What it is

- Two stacks:
 - Encoder** reads the input (bidirectional).
 - Decoder** generates output left-to-right, but with an extra attention mechanism: **cross-attention** to the encoder's outputs.

Used for

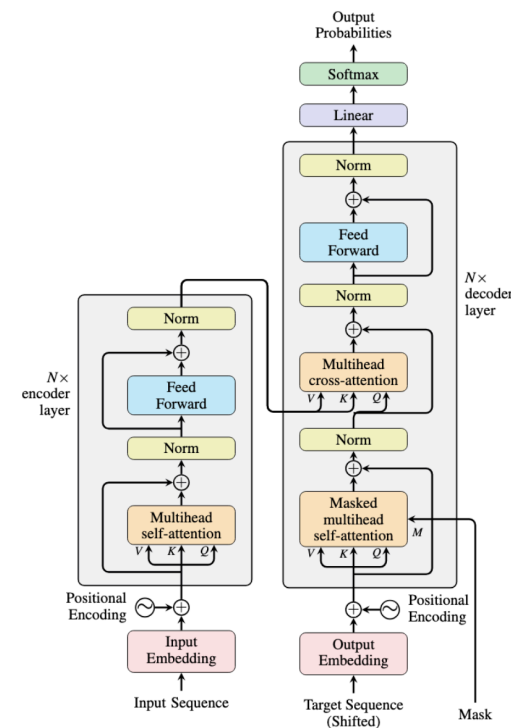
- "Transform one sequence into another" tasks:
 - translation, summarization, rewriting, question answering (generative), structured generation.
- Often better than decoder-only when the output must strongly condition on a provided input.

Discriminative or generative

- Generative** (conditional generation).

Famous examples

- T5, BART, PEGASUS,
- original Transformer (Vaswani et al.),
- MarianMT (translation models).



- conditional allows simplification of probability distribution of the decoder
- preferred over decoder only when context is long, output short

Variants with Prefix casual masking

Objectives, losses and
maximum likelihood

Next-token prediction and cross-entropy

Factorization

A causal language model defines

$$p_{\theta}(x_{1:T}) = \prod_{t=1}^T p_{\theta}(x_t \mid x_{<t}), \quad x_{<t} = x_{1:t-1}.$$

$$p_{\theta}(\cdot \mid \text{context}) = \text{softmax}(z_t).$$

$$H(p, q) = -\mathbb{E}_p[\log q],$$

$$H(p, q) = -\sum_{x \in \mathcal{X}} p(x) \log q(x). \quad (\text{Eq. 1})$$

Since target is a determinist distribution

$$-\sum_{t=1}^T \log p_{\theta}(x_t \mid x_{<t}) \rightarrow \text{Minimizing cross entropy} \leftrightarrow \text{maximize likelihood}$$

Cross-entropy and KL

expected excess surprisal from using the approximation Q instead of P

$$D_{\text{KL}}(P \parallel Q) = \sum_{x \in \mathcal{X}} P(x) \log \frac{P(x)}{Q(x)}.$$

$$H(p, q) = H(p) + D_{\text{KL}}(p \parallel q),$$

Minimize cross entropy <-> minimize KL

Label smoothing

Problem: Real distribution might contain noise (errors, bad cut off etc..)

Idea: replace **hard one-hot labels** with a **soft distribution**

$$q_{\varepsilon}(v) = (1 - \varepsilon)\mathbf{1}[v = y] + \varepsilon \frac{1}{V}$$

So the "correct" class gets probability $1 - \varepsilon + \varepsilon/V$, and every class gets at least ε/V .

- Reduces overconfidence -> specially usefull when you want to encode multiple possibilities (search, beam e
- Regularization → often better generalization
- Improves robustness to mislabeled data

Masked language objective

Train a model to predict missing tokens in a sentence where some tokens are replaced by a special mask

$$\max_{\theta} \sum_{i \in M} \log p_{\theta}(x_i \mid \tilde{x}_{1:T})$$

-> Allows bi-directional attention

Why it's useful

- Learns strong bidirectional representations
- Great for understanding tasks (classification, QA, retrieval) because you want a deep contextual embedding of each token.

Key caveats

- **Train-test mismatch:** the [MASK] token doesn't appear at inference for most tasks.
- Often needs large data and careful corruption strategy or multi-stage training (more complex)

Training pipeline

Training data

How to best make the best of your data? Data curation

Filtering / dedup / contamination / quality heuristics

Dedup at doc + near-dup

Quality filters

Contamination control: remove benchmark/test-like data

Context window management & long-document sampling

Train lengths via a **length schedule** (short → long) to stabilize

Curriculum & “difficulty” scheduling

Don't sample uniformly from day 1 — **stage the learning problem**

Schedules: quality-first → diversity; short → long; easy → hard; domain ramp (code/math ↑)

Backpropagation

$$\frac{\partial \mathcal{L}}{\partial W} = \frac{\partial \mathcal{L}}{\partial y} \frac{\partial y}{\partial W}$$

How to compute gradients for very big nested functions like lms?

Solution autograd: Autograd in modern deep learning is basically **reverse-mode automatic differentiation** (backprop through a computation graph)

it builds a **Directed Acyclic Graph (DAG)** that tracks all operation

AD applies the chain rule **backwards** through the recorded ops

Optimizers

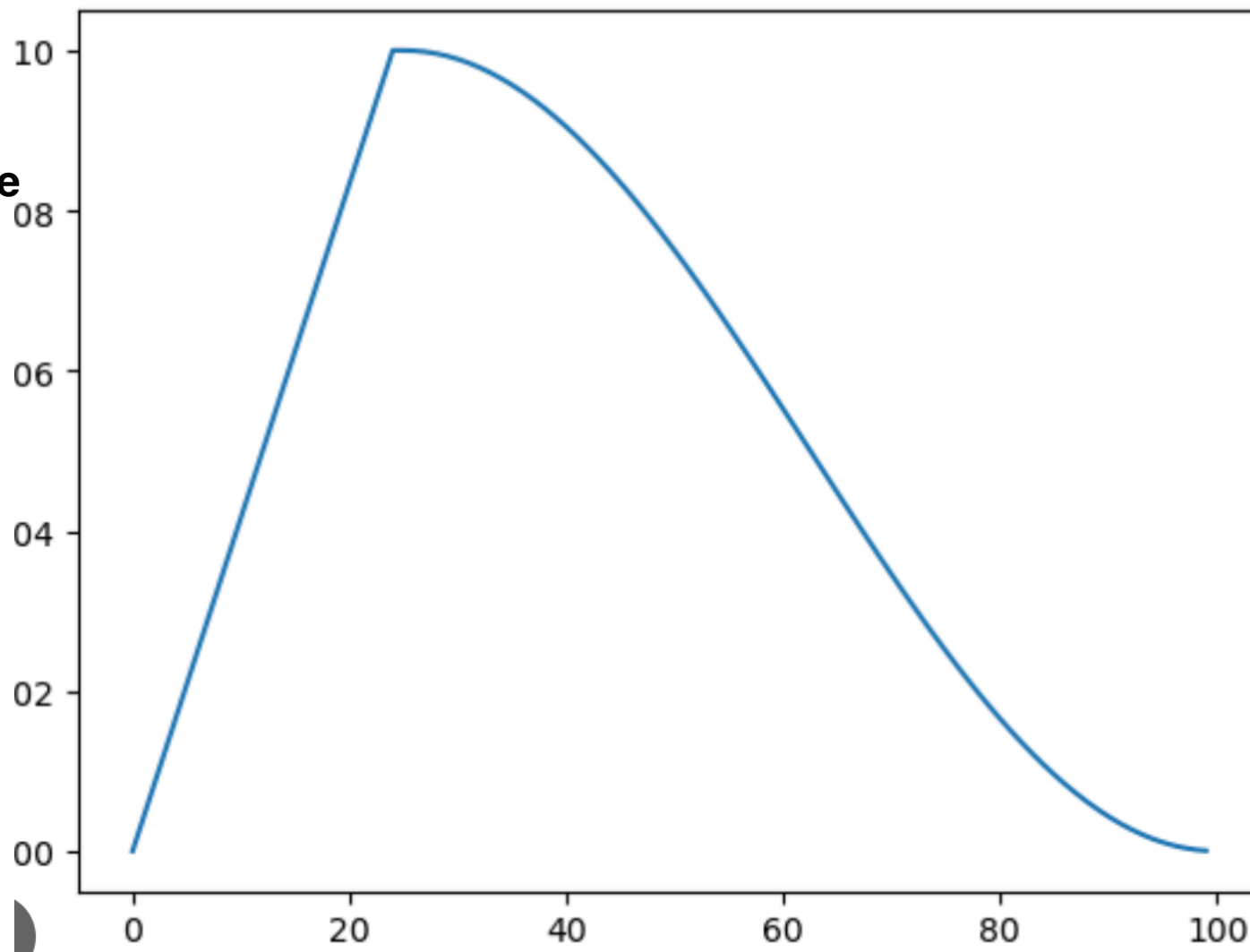
Optimizer	State	Update formulas	Pros	Cons / gotchas
SGD	—	$\theta_t = \theta_{t-1} - \eta g_t$	Simple, low memory; can generalize well with a good LR schedule	Sensitive to LR; can be slow on ill-conditioned problems
SGD + Momentum	v	$v_t = \mu v_{t-1} + g_t ; \theta_t = \theta_{t-1} - \eta v_t$	Faster & smoother than SGD; helps in ravines	Need to tune μ (often 0.9) + LR schedule
AdaGrad	r	$r_t = r_{t-1} + g_t^2 ; \theta_t = \theta_{t-1} - \eta \frac{g_t}{\sqrt{r_t + \epsilon}}$	Great for sparse features; automatic per- parameter step sizes	r_t grows forever → LR shrinks a lot; can stall on deep nets
RMSProp	r	$r_t = \rho r_{t-1} + (1 - \rho) g_t^2 ; \theta_t = \theta_{t-1} - \eta \frac{g_t}{\sqrt{r_t + \epsilon}}$	Fixes AdaGrad "stall" by using a moving average; good for non-stationary loss	Extra hyperparam ρ (often ~0.9); not always best generalization
Adam	m, v	$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t ; v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 ;$ bias-correct $\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \hat{v}_t = \frac{v_t}{1 - \beta_2^t} ; \theta_t = \theta_{t-1} - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t + \epsilon}}$	Strong default; fast early progress; robust to gradient scaling	Can generalize worse than SGD+mom in some setups; still benefits from LR schedule/warmup

warmup

prevents **early instability/divergence**

Use when loss spikes in first
few hundred/thousand steps

Specially useful for adams where
initially states are not yet well
formed

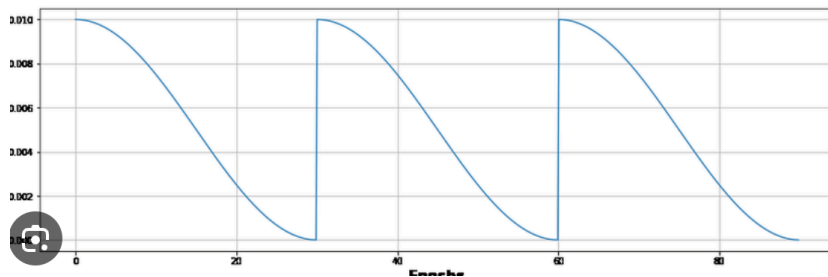


Learning rate schedulers

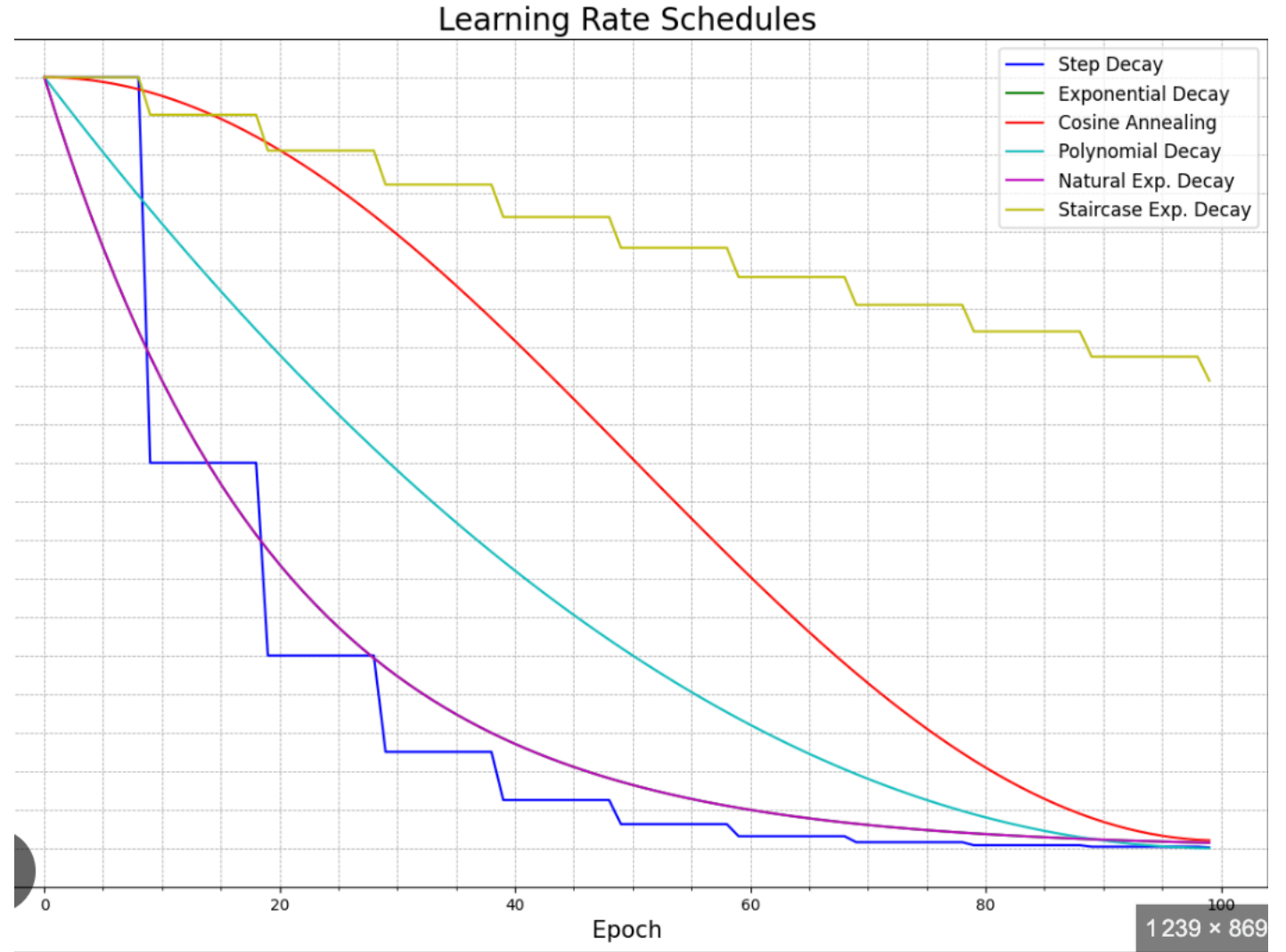
Cosine scheduler:
decays **slowly at first** (allows good exploration) and
faster near the end (good refinement)

Avoids local minima traps at the beginning
and allows precision once in good spot

->yields strong results “out of the box”



With plateau, cosine with cycle can
Help escape local minimum

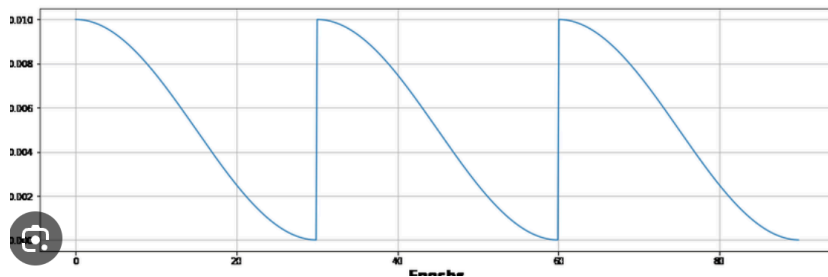


Learning rate schedulers

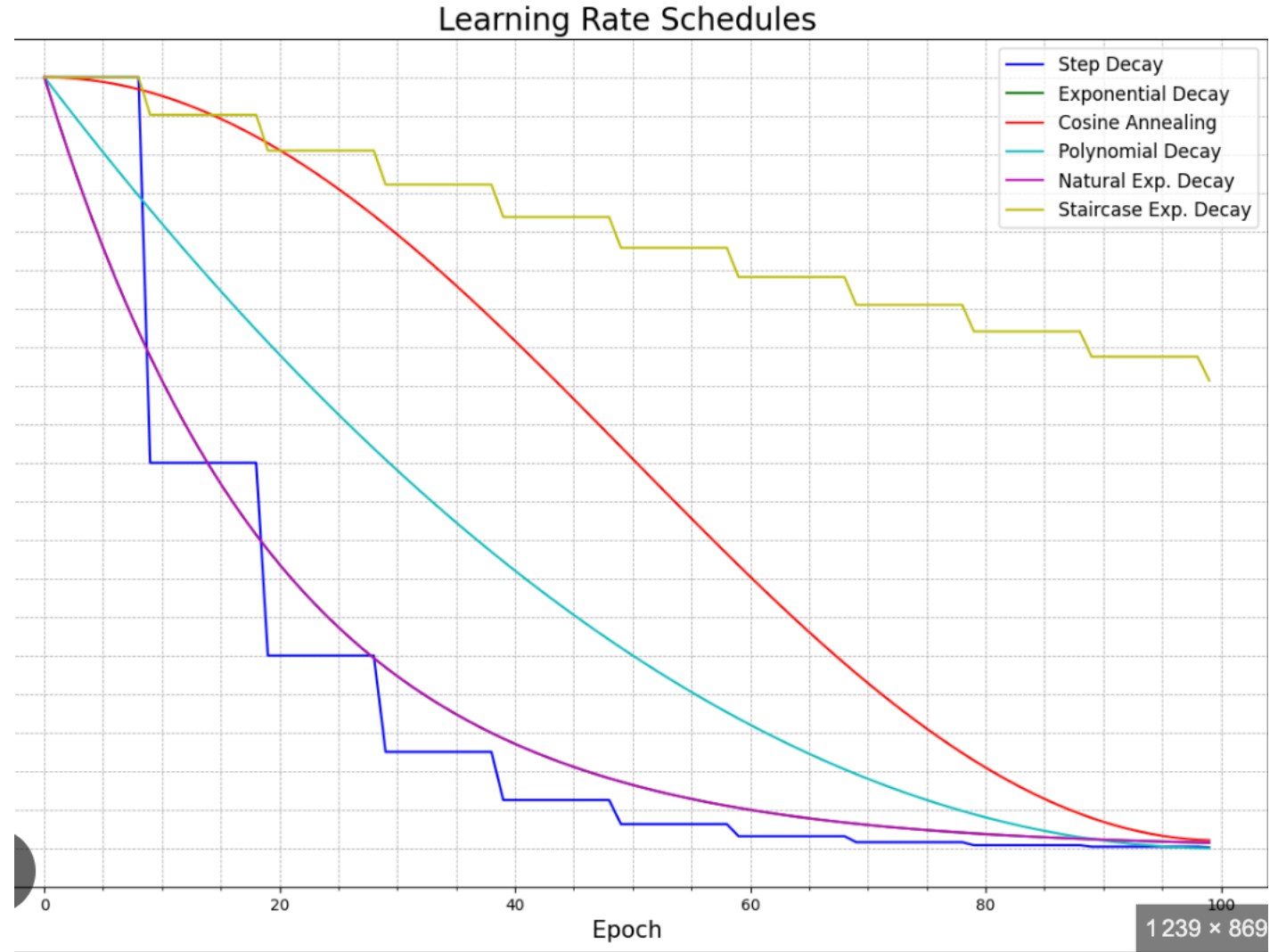
Cosine scheduler:
decays **slowly at first** (allows good exploration) and
faster near the end (good refinement)

Avoids local minima traps at the beginning
and allows precision once in good spot

->yields strong results “out of the box”



With plateau, cosine with cycle can
Help escape local minimum



Gradient clipping

What it is: cap the size of gradients (or the update) so one bad step can't blow up training.

$$g \leftarrow g \cdot \min \left(1, \frac{c}{\|g\|} \right)$$

If norm of gradient $> c \rightarrow$ max it out

- Prevents rare **gradient spikes** (esp. early training, mixed precision, long sequences)

Batching and global batch

Why batching exists

- Improves throughput by parallelizing matrix ops on GPU/TPU
- Reduces gradient noise vs single-example updates

Three related terms

Microbatch or per gpu batch size: what fits in GPU memory per forward/backward

Gradient accumulation: sum gradients over multiple microbatches before stepping

Global batch size: effective batch used per optimizer update across all devices

$$\text{GBS} = (\text{microbatch size}) \times (\text{accumulation steps}) \times (\text{\#devices})$$

Bigger batches Pro:

- smooth out gradients (better loss estimation)
- Better hardware throughput
- Often allows larger learning rates

smaller batches Pro:

- More frequent parameter updates (better for adam like)
- Easy to fit in memory
- More gradient noise can act like regularization

-> **sweet spot:** the largest batch that still improves (or at least doesn't hurt) validation for your compute bu

Weight decay and l2 penalty

What it is: a regularizer that continuously pulls weights toward zero -> encourages model sparsity -> more generalization

Standard L2 penalty

$$\mathcal{L} + \frac{\lambda}{2} \|\theta\|^2$$

Same for SGD but not for adaptive algos like adam

Decoupled weight decay (preferred)

$$\theta \leftarrow (1 - \eta\lambda) \theta$$

$$\theta \leftarrow \theta - \eta \frac{(g_t + \lambda\theta)}{\sqrt{v_t} + \epsilon}$$

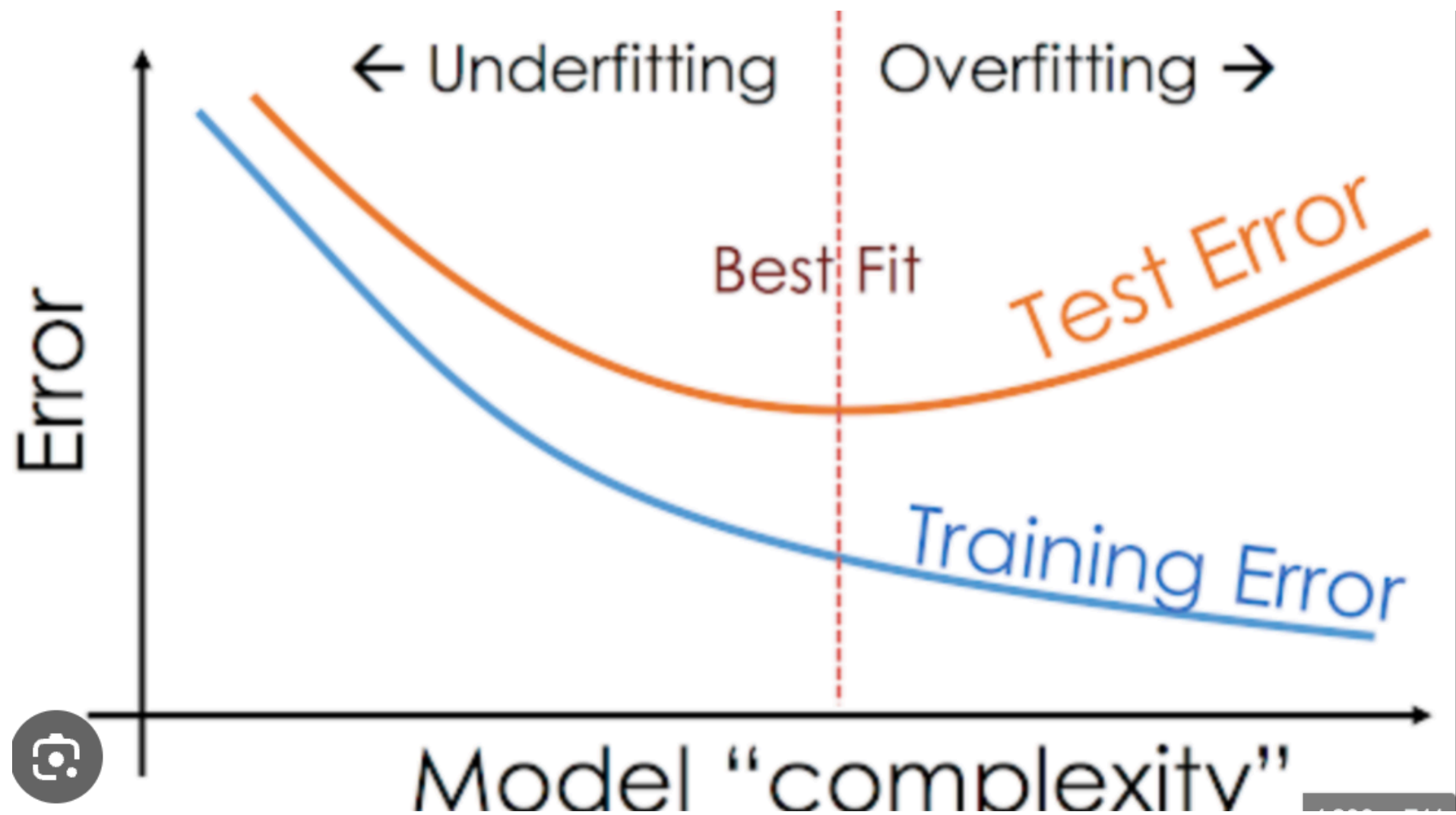
So the "decay" is **not uniform** anymore. Parameters with large v_t get **less** shrink, and ones with small v_t get **more** shrink.

Why it helps

Reduces overfitting, improves generalization

Keeps weights from growing too large (helps stability)

Overfitting-underfitting



Exercises

In presence of overfitting, what would you do?

In presence of underfitting, what would you do?

Efficient training compute

Why we need it

Training is usually **memory-bandwidth bound** (not pure FLOPs), especially for attention.

Memory limits determine **max model size / batch size / sequence length**.

Optimizer states + activations dominate memory; attention can dominate both memory and time at long context.

Efficient compute directly translates to:

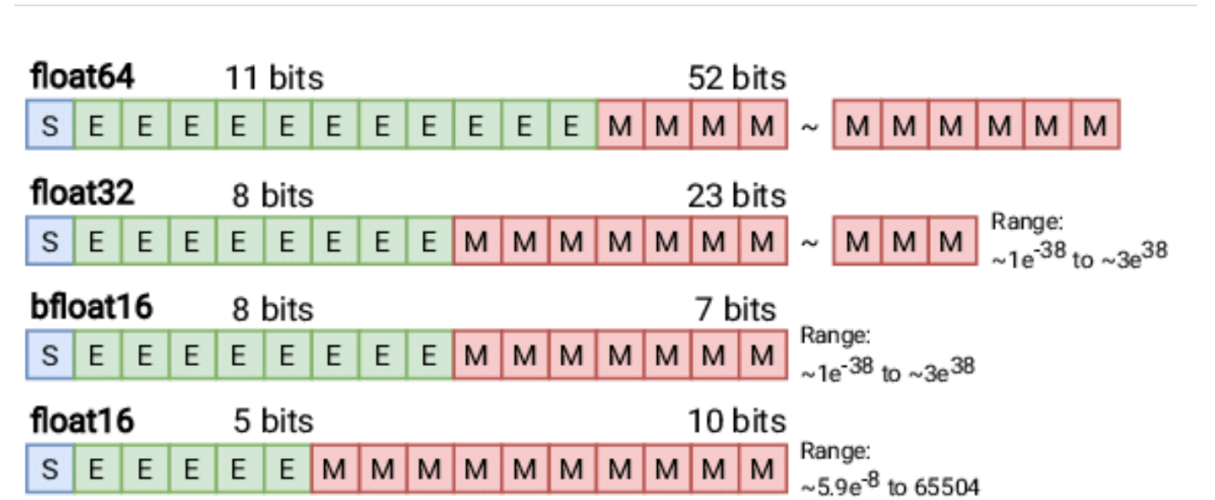
- faster iteration (more experiments/week)
- cheaper training (\$/run)
- ability to scale context + model safely

Half-precision

Storing and computing objects in half precision (FP16) divides
By two the memory foot print

Reduced range -> overflow

Too small values -> underflow



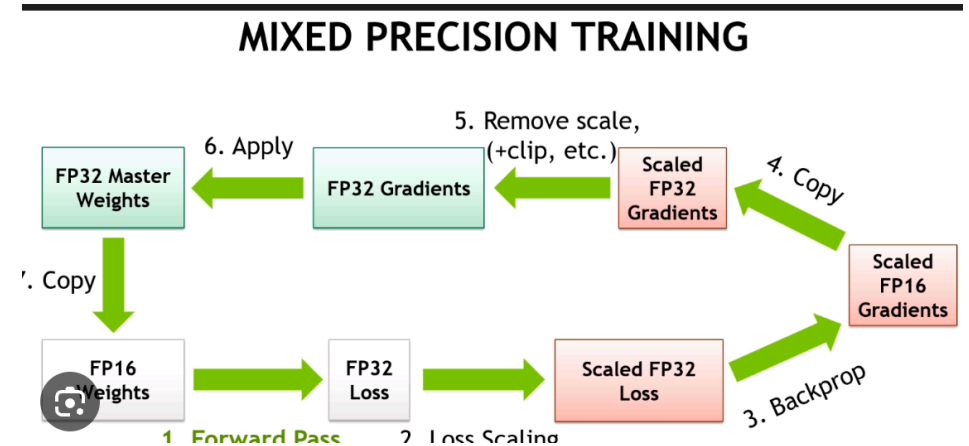
Problems

- Gradients often tend to be small due to chainrule (underflow)
- updates $lr \cdot \text{gradient}$ -> can be very small (underflow)
- Use of exp in softmax in loss computation -> can be high (overflow)
- Mathematical steps that involve accumulation (activations,attention scores, norm, maltnut) can give high values (overflow)

the grid is *dense near 0* and *sparse as numbers get large*

Mixed-precision

- **GEMMs (Linear / QKV / MLP):** multiply in **bf16/fp16**, accumulate in **fp32**, outputs usually **bf16/fp16**
- **Softmax + LayerNorm (reductions):** computed with **fp32** internal stats (max/sum, mean/var) for stability
- **Loss (scalar):** treated as **fp32**
- **Backward:** uses the same rule—big matmul grads use **bf16/fp16 multiply + fp32 accumulate**, sensitive reductions (sum, mean etc..) use **fp32**
- **Optimizer (Adam):** optimizer states + updates in **fp32** (often with **fp32 master weights**)
- **FP16 only:** add **loss scaling** (scale loss → backprop → unscale grads, check overflow)



Note: in practice advanced hardware like tensor cores already smartly handle matmul in mixed precisions (dot in fp16/bf16, acc in fp32)

ACTIVATION CHECKPOINT

- **Activation checkpointing (gradient checkpointing):** save GPU memory by **not storing all intermediate activations** in the forward pass; **recompute them during backward**.
- **Why needed:** activation memory can become very large -> Enables **bigger batch / longer context / larger models**.
- **How it's done:** store only **checkpoint tensors** (typically transformer block inputs); in backward, **re-run forward** for that block/segment to reconstruct activations.
- **Tradeoff:** **memory ↓ a lot, compute ↑** (often ~15–40% slower step time, depends on checkpoint granularity). -> trade memory for compute

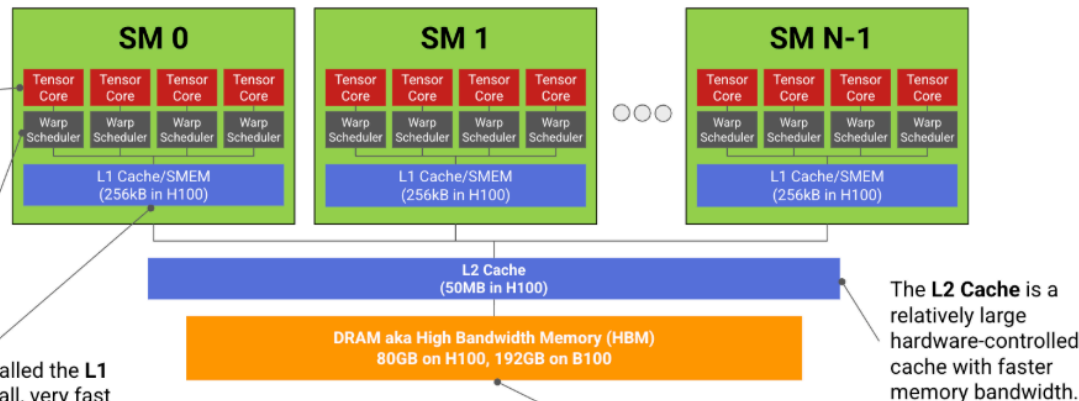
Fused kernels

- **Fused kernels** = implementing a *sequence of operations* as **one GPU kernel** instead of many separate kernels.
- On GPUs, Transformers are often **memory-bandwidth bound** for many ops (LayerNorm, softmax, bias+activation, residual).
- If we run these as separate kernels, you repeatedly:
 - 1:read a big tensor from HBM (GPU memory)
 - 2:write an intermediate back to HBM
 - 3:launch another kernel to read it again
-

The **Tensor Core** performs matrix multiplications and accounts for most of the chip FLOPs/s, similar to TPU MXU

The **Warp Scheduler** is a SIMD vector unit like the TPU VPU with 32 lanes, called "CUDA Cores". All lanes must perform the same operation in each cycle.

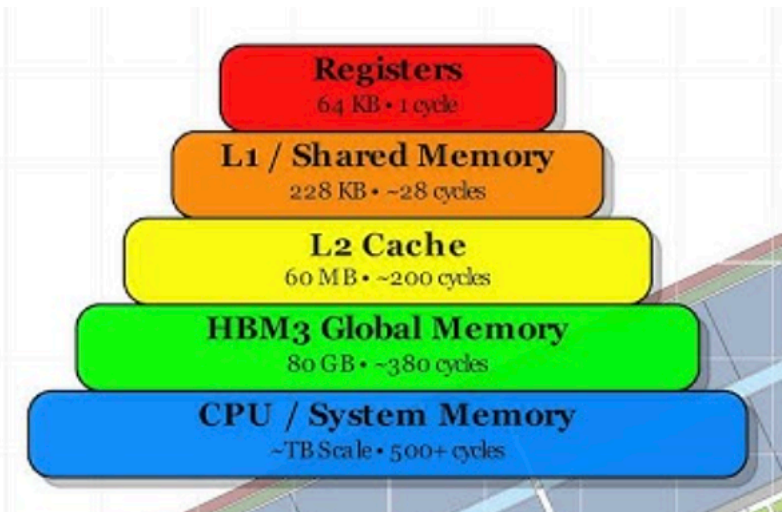
SMEM (often called the **L1 Cache**) is a small, very fast on-chip cache that can be programmer controlled. Similar to TPU VMEM but much smaller.



Abstract layout of an H100/B100 GPU

DRAM or **HBM** stores parameters, activations, optimizer state, etc.

1010 x 888



Fused LayerNorm

What LayerNorm needs to compute

For each token (each position), you have a hidden vector $x \in \mathbb{R}^d$ (e.g., $d = 4096$). LN computes per-token stats:

$$\mu = \frac{1}{d} \sum_{i=1}^d x_i, \quad \sigma^2 = \frac{1}{d} \sum_{i=1}^d (x_i - \mu)^2, \quad rstd = \frac{1}{\sqrt{\sigma^2 + \epsilon}}$$

$$y_i = (x_i - \mu) \cdot rstd \cdot \gamma_i + \beta_i$$

Key point: μ and σ^2 are computed across the feature dimension d for each token.

Unfused

Kernel launches (typical): 3 kernels

- reduce for mean (Read: x , write: μ)
- reduce for **variance** (read: x , μ write σ)
- **Normalize + affine** (read: x , μ , σ , write: output)

HBM traffic

Reads: 3 reads of x , 2x μ , 1x σ

Writes: 1x μ , 1x σ , 1x output

Why slower: memory-bandwidth bound
+ extra launch/sync overhead

Fused LayerNorm (single fused CUDA kernel)

Kernel launches (typical): 1

one kernel does **mean + var + normalize + affine** in one go
(stats kept on-chip as much as possible)

HBM traffic (rough, per element of x):

Reads: $\sim 1 \times x$

Writes: $1 \times$ output

(mean/var usually *not* spilled to HBM in forward unless needed for backward/debug)

Why faster: far fewer HBM round-trips + fewer kernel launches.

Flash attention

For attention:

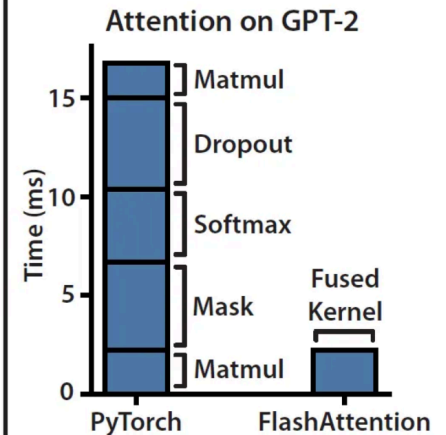
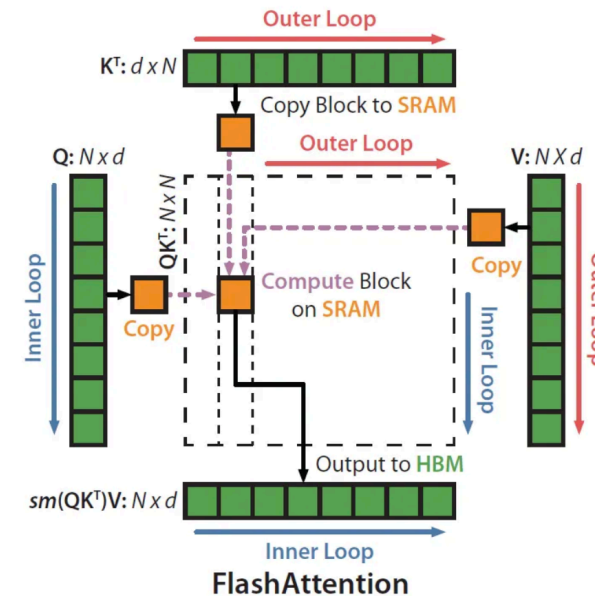
$$S = \frac{QK^T}{\sqrt{d}} + \text{mask}, \quad P = \text{softmax}(S), \quad O = PV$$

Many kernel launches

-> Let's fuse operation

It computes attention in **tiles/blocks** that fit in fast on-chip memory (SRAM/shared memory):

1. Take a block of queries Q_{block}
2. Stream blocks of keys/values $K_{\text{block}}, V_{\text{block}}$
3. Compute partial scores and apply **online softmax**:
 - keep running per-row **max** and **normalization** across blocks
4. Accumulate the output for that query block incrementally
5. Write only the final output to HBM

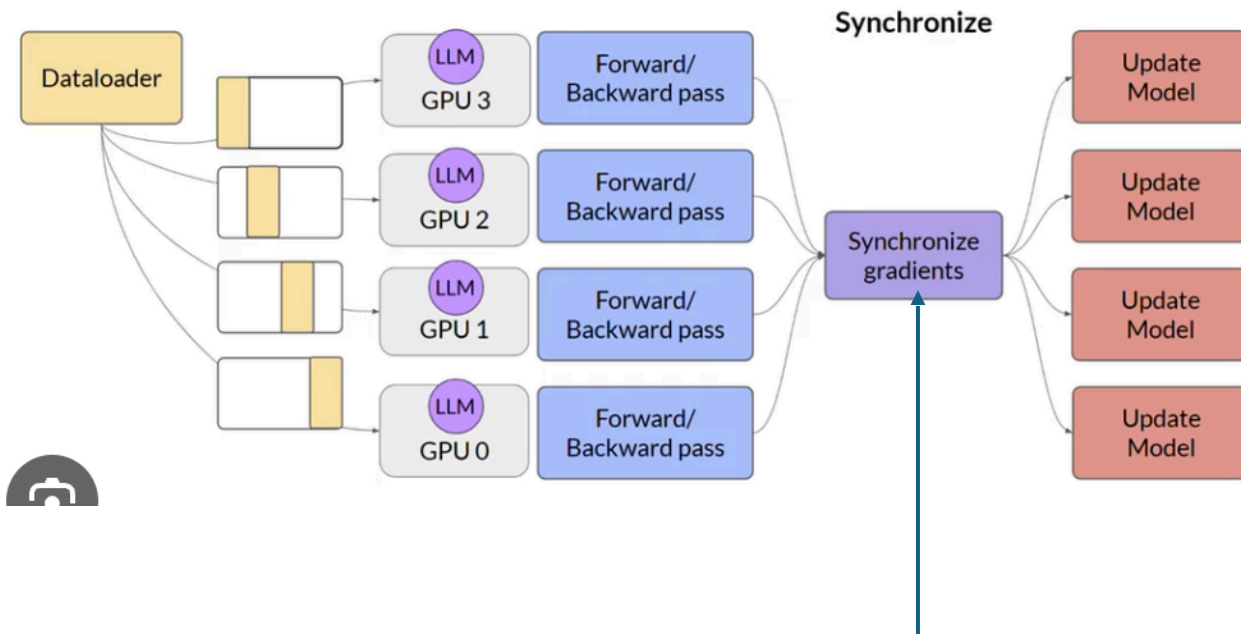


No intermediate savings (in particular of the big A (LxL)

- Reduction in HBM reads/writes -> better bandwidth communication -> faster
- No write of full A -> memory reduction
- Exact attention (no approximation

Distributed training systems

DDP (distributed data parallel)



All reduce

Bigger effective batch size -> faster training

main bottlenecks

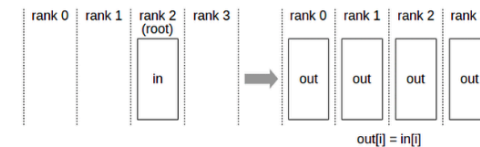
- **Gradient synchronization** cost (all-reduce bandwidth/latency), especially at many GPUs.
- **Small batch / small model** $\Rightarrow$ comm dominates compute.
- Requires gpu to fit inside each device

Model fit inside each gpu

All reduce



Broadcast



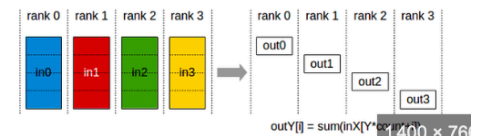
Reduce



All Gather



Reduce Scatter



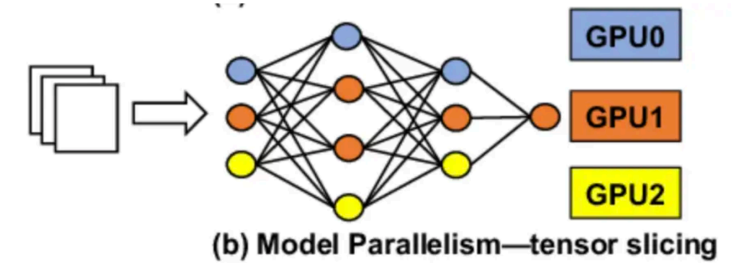
- **all-gather** when outputs are *concatenated pieces*
- **all-reduce** when outputs are *summed contributions*

Tensor / model parallelism (TP)

What it is

Shard **within a layer** (e.g., split weight matrices / attention heads across GPUs).

Each forward/backward needs **collectives** to combine partial results (all-gather / reduce-scatter / all-reduce).



When to use

Model **doesn't fit** on one GPU (parameters or activations).

You want to scale **matmul-heavy layers** across multiple GPUs per layer. (**wide models**)

Often combined with DDP

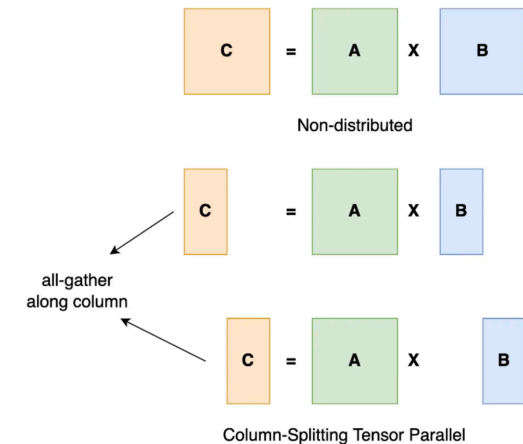
More common during inference

Main bottlenecks

Communication every layer (frequent, latency-sensitive).

Requires **fast interconnect**; otherwise comm kills speed.

Harder to keep kernels efficient (shapes/partitioning), more complex debugging/perf tuning.



Pipeline parallelism (PP)

What it is

Split the model into **stages** (groups of layers) across GPUs.

Run **microbatches** in a pipeline: while stage 0 runs microbatch $k+1$, stage 1 runs k , etc.

When to use

Model is **too deep (many layers)** to fit on one GPU, and you prefer sharding by layers (clean for Transformers).

Often combined with TP + DDP (3D parallelism).

More common during inference

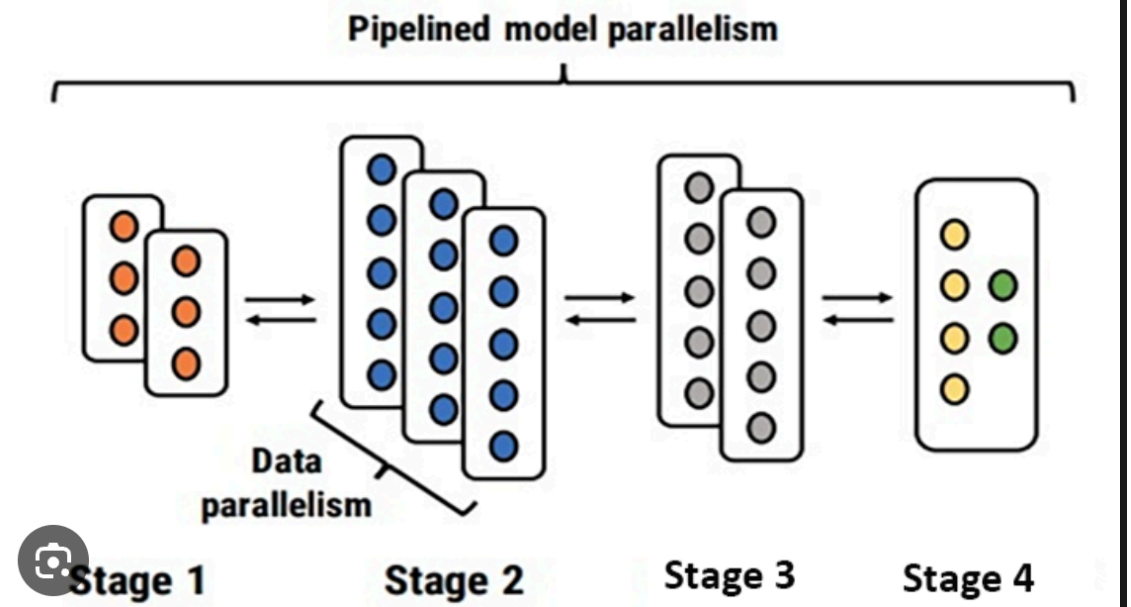
Main bottlenecks

Pipeline bubble (idle GPUs at start/end; worse with few microbatches).

Stage imbalance (slowest stage sets throughput).

Activation memory grows with #microbatches (unless you recompute/checkpoint).

Inter-stage **activation communication** and scheduling complexity



Stages exchange activations/gradients with **point-to-point sends/recvs**

Fully sharded data parallel

What can we do when the memory of the objects during training is far bigger than the model params itself?

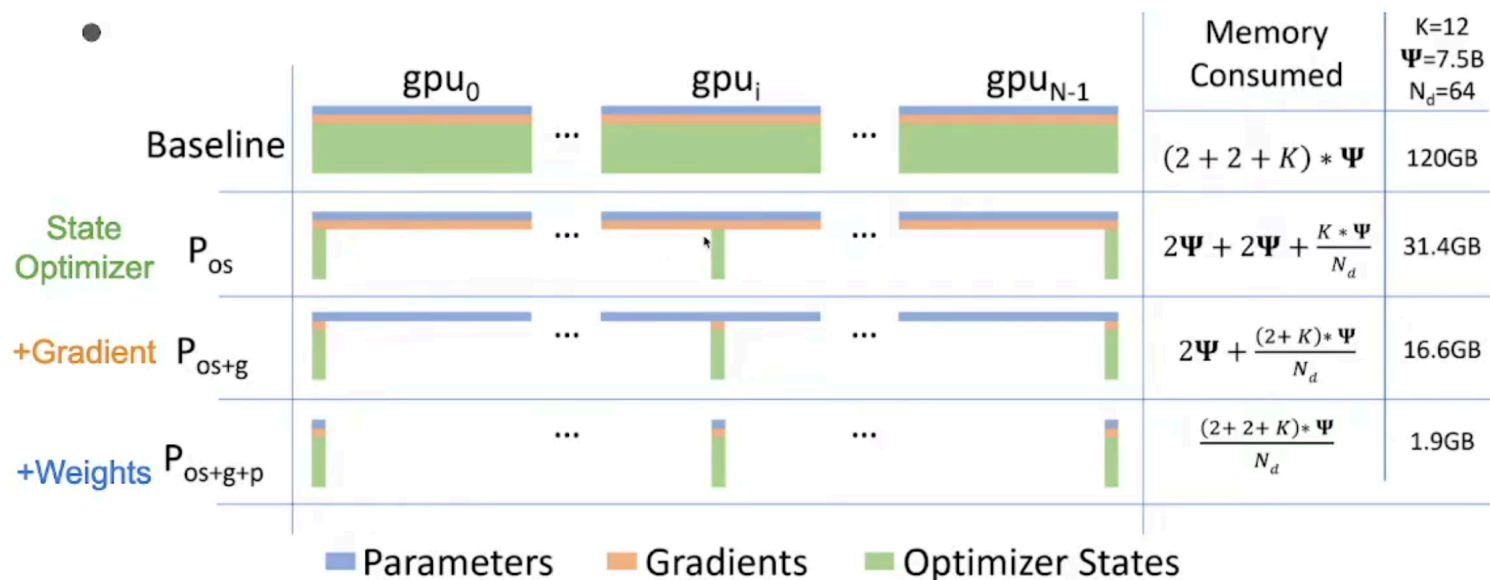
Assuming FP16 and x parameters (*all trainable*)

- Parameters $\Rightarrow 2x$ (2 bytes for float16)
- Gradients $\Rightarrow 2x$
- Optimizer State [Adam] $\Rightarrow 12x$
 - Parameter copy $4x$ (4 bytes for float32)
 - Momentum $4x$
 - Variance $4x$
 - All stored in float32!!

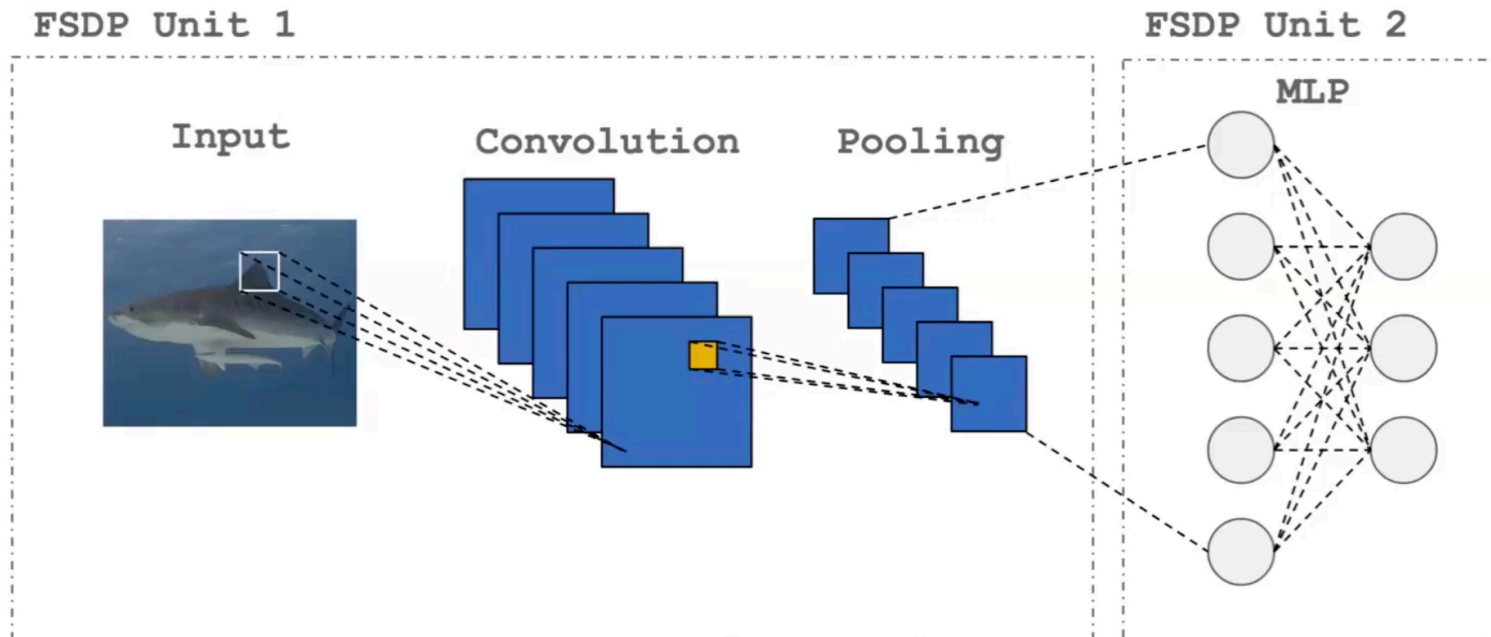
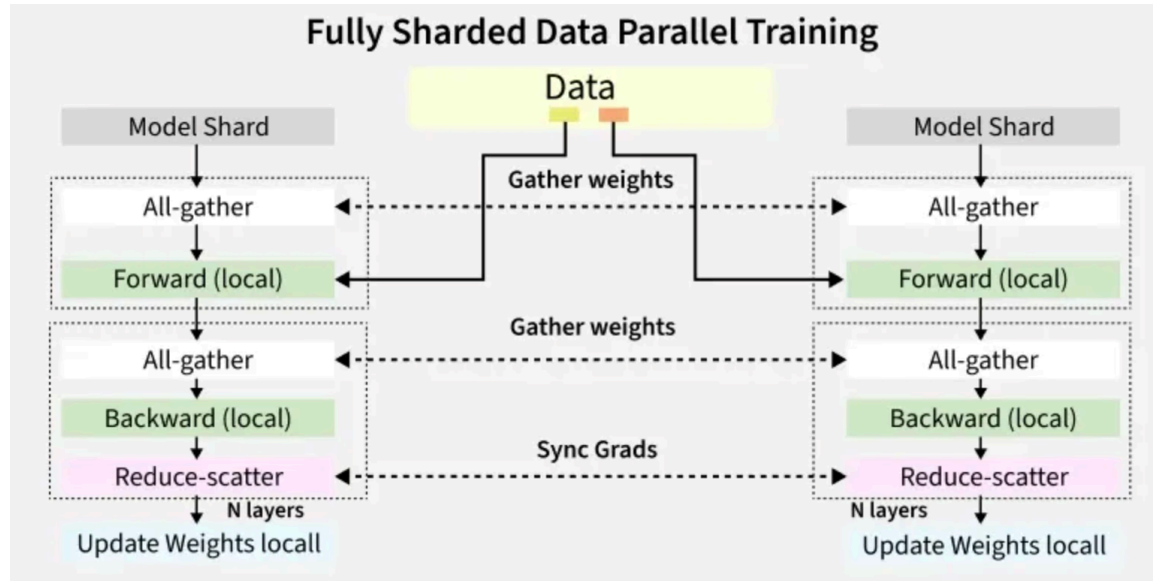
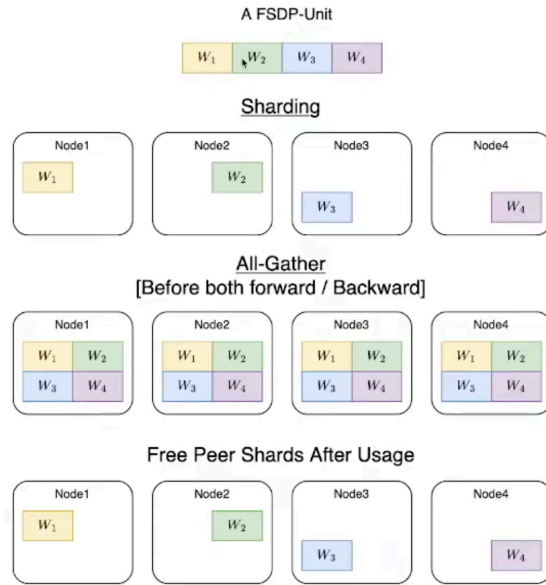
->solution: SHARD THEM+DDP

- ZeRO-1:** shard **optimizer states**
- ZeRO-2:** shard optimizer states + **gradients**
- ZeRO-3:** shard optimizer states + **gradients + parameters** (full sharding)

Note: data is also shared DDP style



Fully shared data parallel



Excercises

Suppose we have a 2 layer transformer, with one FSDP unit per layer, suppose we have 8 gpus (1 node)

Questions 1 in zero-1 (only optimizer states) how many and which collectives needs to be called during a full update cycle?

Questions 2 in zero-2 (Grads and optimizer states) how many and which collectives needs to be called during a full update cycle?

Questions 3 in zero-3 (Grads and optimizer states and model weights) how many and which collectives needs to be called during a full update cycle?

MoE mixture-of-experts

For a token hidden state h :

- Router produces scores:

$$s = W_r h$$

- Choose **top-k** experts (commonly $k=1$ or 2):

$$\mathcal{E} = \text{TopK}(\text{softmax}(s))$$

- Output is a weighted sum of expert outputs:

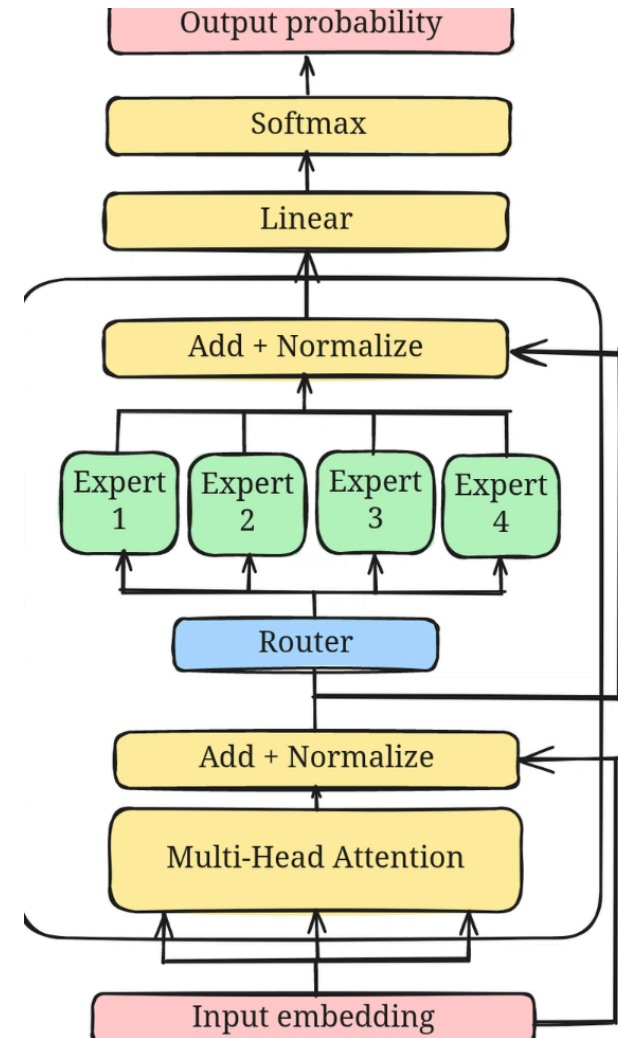
$$y = \sum_{e \in \mathcal{E}} p_e \cdot f_e(h)$$

where f_e is expert e 's FFN, and p_e are router weights.

- **Scale parameters cheaply:** more capacity without proportional FLOPs.
- Often improves quality at similar compute budgets (esp. at large scale).
- Enables “sparse” scaling: **more model, not much more compute per token**
- .

Let $F = \{f_1, f_2, \dots, f_n\}$ represent the proportion of tokens assigned to each expert, where f_j is the number of tokens assigned to the j -th expert. For any given MoE layer, the total loss function $\mathcal{L}$ consists of two parts, the main loss $\mathcal{L}_h$ and the auxiliary loss $\mathcal{L}_{aux}$:

$$\mathcal{L} = \mathcal{L}_h + \alpha \cdot \mathcal{L}_{aux} = \mathcal{L}_h + \alpha \sum_{j=1}^n f_j \cdot p_j, p_j = \sum_{i=1}^N s_{ij}, \quad (2)$$



Advancing Expert Specialization for Better MoE

- Paper: <https://openreview.net/forum?id=iydmH9boLb>
- Observation: commonly used auxiliary load balancing loss often leads to expert overlap and overly uniform routing
- Idea: introduce a new loss that combines two complementary objectives:
 - (1) an orthogonality loss to encourage experts to process distinct types of tokens, and
 - (2) a variance loss to encourage more discriminative routing decisions

Expert Specialization. We introduce an orthogonalization objective that encourages independent expert representations. Specifically, we design the following orthogonality loss:

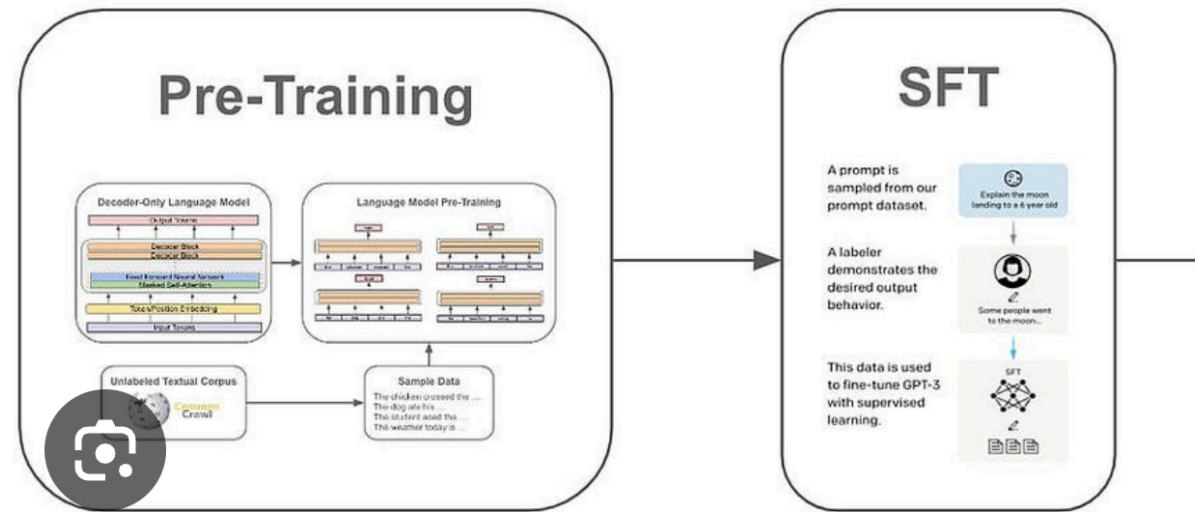
$$\mathcal{L}_o = \sum_{i=1}^N \sum_{j=1}^n \sum_{\substack{k=1 \\ k \neq j}}^n \frac{\langle \tilde{x}_{ij}, \tilde{x}_{ik} \rangle}{\langle \tilde{x}_{ik}, \tilde{x}_{ik} \rangle} \tilde{x}_{ik}, \tilde{x}_{ij} = x_i \cdot \theta_{E_j} \cdot \mathbb{I}_{\{s_{ij} > 0\}}, i \in [1, N], j \in [1, n], \quad (6)$$

Routing Diversification. We introduce a variance-based loss to encourage more diverse routing decisions and promote expert specialization. Specifically, we define the variance loss as:

$$\mathcal{L}_v = - \sum_{i=1}^N \sum_{j=1}^n \frac{1}{n} \cdot (s_{ij} - \bar{s}_j)^2, \bar{s}_j = \frac{1}{N} \cdot \sum_{i=1}^N s_{ij}, \quad (7)$$

Post-training / adaptation methods

Supervised finetuning



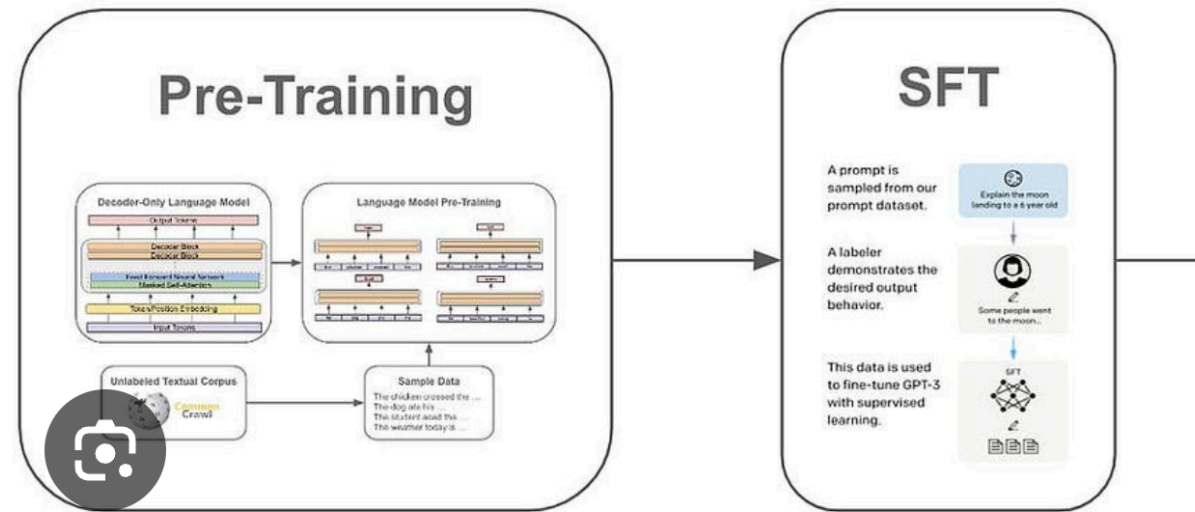
What it is: Train the model on **labeled (input → target)** examples (e.g., instruction → ideal answer) using standard **cross-entropy** / teacher forcing.

Goal: Make the model **follow instructions**, match a desired style/format, and improve task performance on the supervised dataset.

When to use: You have **high-quality demonstrations** and clear targets (good for adding capabilities, domain adaptation, formatting).

Main gotchas: Can **overfit** narrow patterns, **forget** general skills (catastrophic forgetting), and may not align with human preferences beyond what's in the labels.

Supervised finetuning



What it is: Train the model on **labeled (input → target)** examples (e.g., instruction → ideal answer) using standard **cross-entropy** / teacher forcing.

Goal: Make the model **follow instructions**, match a desired style/format, and improve task performance on the supervised dataset.

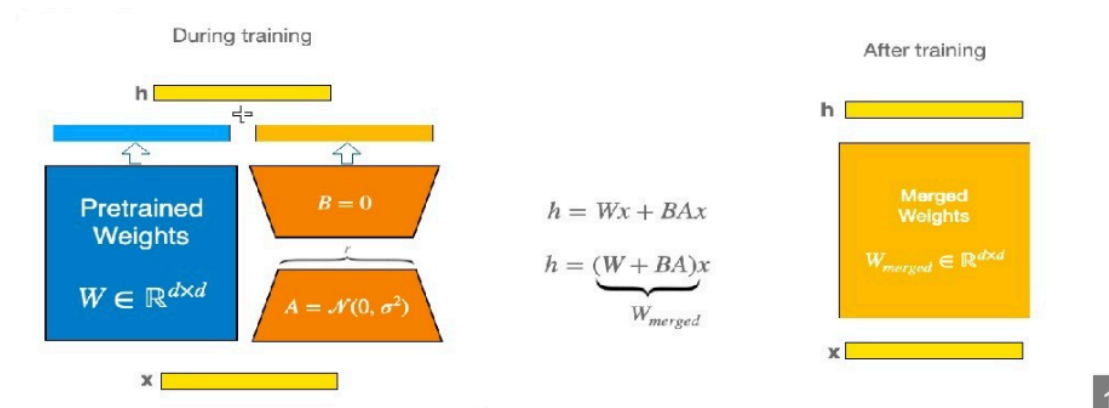
When to use: You have **high-quality demonstrations** and clear targets (good for adding capabilities, domain adaptation, formatting).

Main gotchas: Can **overfit** narrow patterns, **forget** general skills (catastrophic forgetting), and may not align with human preferences beyond what's in the labels.

LoRA (Low-Rank Adaptation)

full fine-tuning impractical for most real-world applications

-> **LoRA (Low-Rank Adaptation)** solves this problem by **not updating all model weights**.



- Full: $4096^2 = 16,777,216$ params
- LoRA: $8(4096 + 4096) = 65,536$ params
→ ~256x fewer trainable params

- Reduced memory footprint
- faster

Variants: **QLoRA: LoRA + Quantization**

QLoRA (Quantized Low-Rank Adaptation) combines the best of both worlds

The **base model is quantized** (e.g., 4-bit)

LoRA adapters are trained on top of it

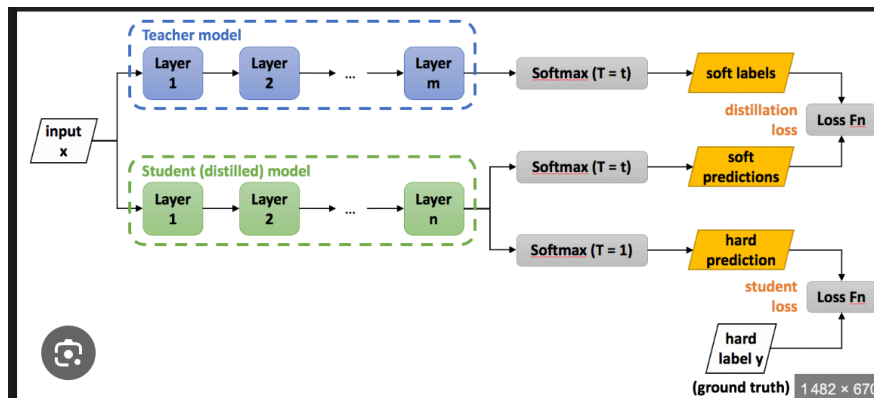
Distillation

Train a **smaller “student”** model to match a **larger “teacher”** model’s behavior, so you keep most quality while reducing cost.

how to build the student? Fewer layers (reduce depth), reduced hidden_dim D_model , reduced FNN expansion D_FF

- **Smaller models:**
- **Faster inference / lower latency:**
- **Lower compute & cost:**
- **Easier deployment:**

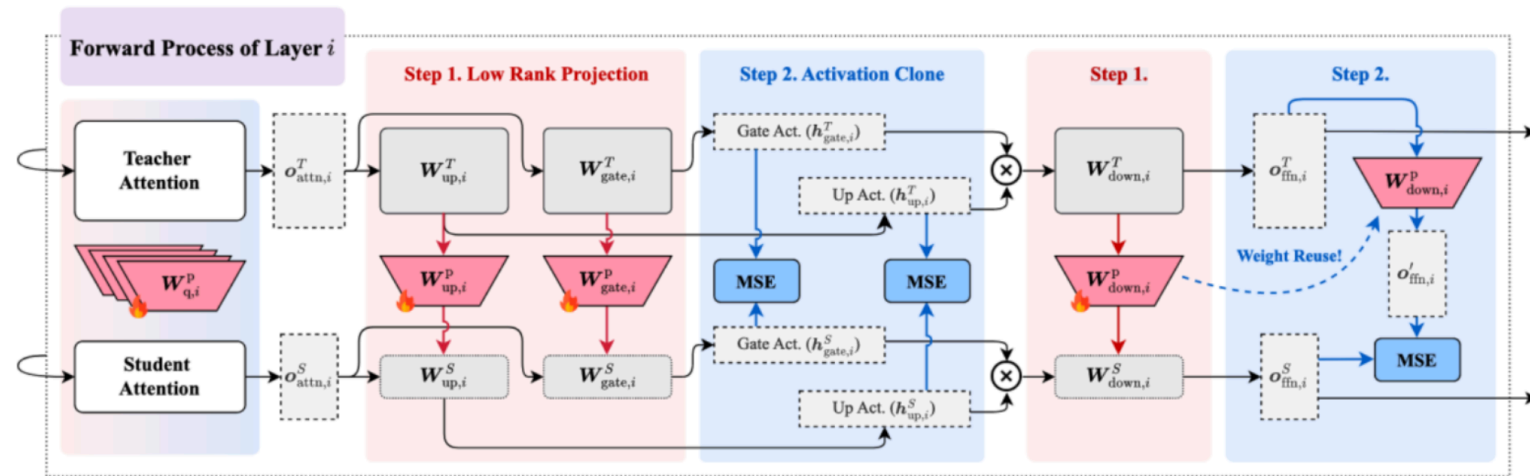
Knowledge distillation: the student model is trained using the teacher model's output probabilities, known as *soft targets*, alongside the ground truth labels, referred to as *hard targets*



$$\mathcal{L} = \alpha \mathcal{L}_{KD} + (1 - \alpha) \mathcal{L}_{CE}$$

Efficient Knowledge Distillation through Low-Rank Clone

- Paper: <https://openreview.net/pdf?id=LVDJRJE4xQ2>
- Key ideas : 1) learn a low-rank projection matrices W_p to compress teacher weights W_T to obtain student
- Weights $W_S = W_T * W_p$
- 2) clone activations and penalize strong deviations in the loss

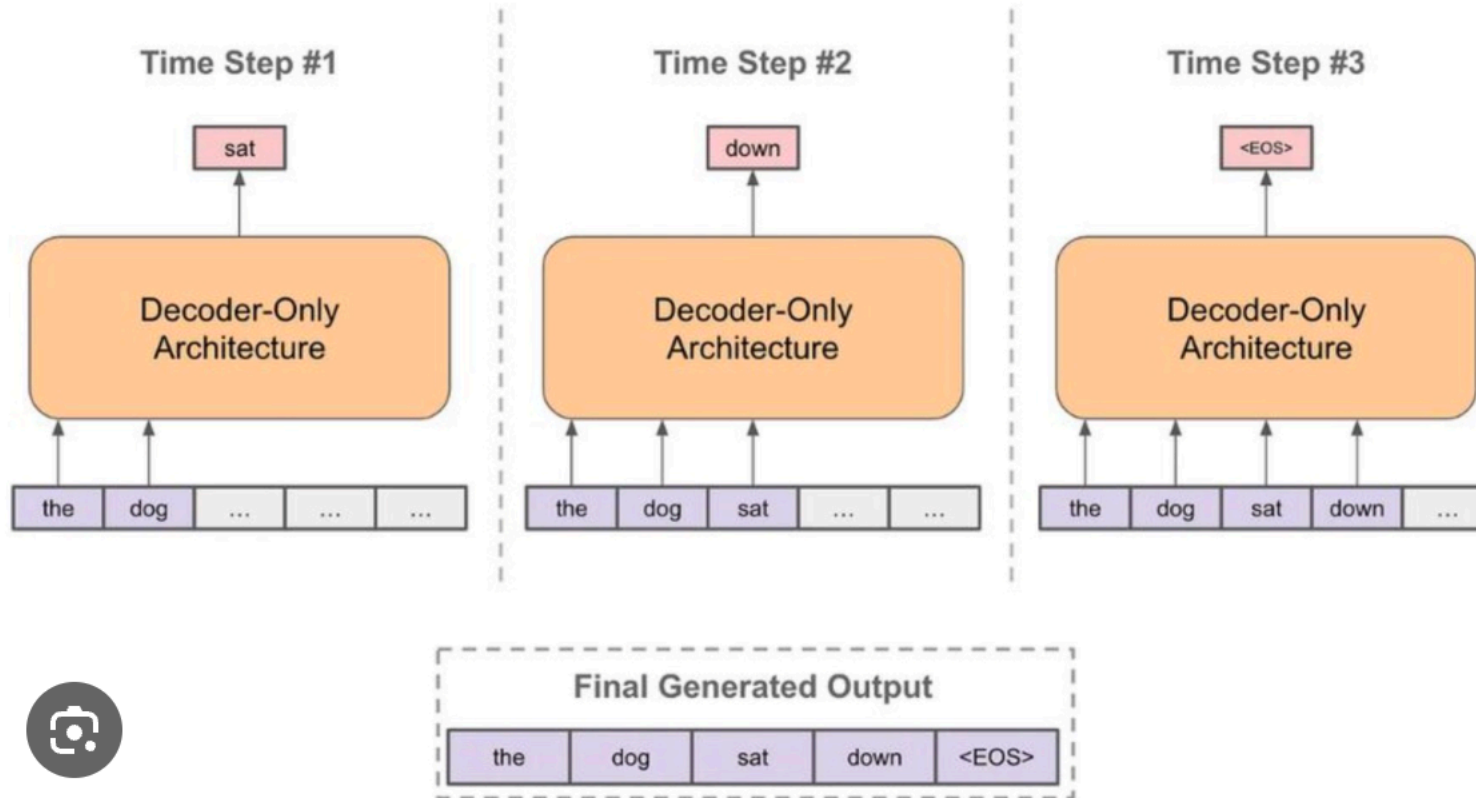


Training loss to preserve teacher information as much as possible

$$\mathcal{L} = \mathcal{L}_{\text{KL}} + \mathcal{L}_{\text{LM}} + \alpha \mathcal{L}_{\text{clone}},$$

Inference and decoding

Autoregressive Decoding



How to select the token at each step?

Decoding strategies

Greedy decoding

Rule: pick the highest-prob token each step: $x_t = \operatorname{argmax} p(\cdot)$

Pros: fastest, deterministic, good for short/simple tasks

Cons: can get stuck in local choices; lower global coherence; less diverse

Sampling

- Sample from the logits

Temperature

Scale logits: $p \propto \operatorname{softmax}(z/T)$

Lower T: more deterministic; **higher T:** more random

Top-k sampling

Sample only from **k** most likely tokens

Controls “tail risk” (removes very unlikely tokens)

Top-p / nucleus sampling

Sample from smallest set whose cumulative prob $\geq p$

Adapts set size to uncertainty (often better than fixed k)

Pros of sampling (temperature / top-k / top-p etc.)

- Diversity & creativity
- Escapes greedy failures

Cons of sampling

- Less reliable / less factual
- Non-deterministic

Beam search

Beam search

Rule: keep top **B** partial sequences (“beams”), expand, keep best by total log-prob

$$\max \sum \log p(x_t | x_{<t})$$

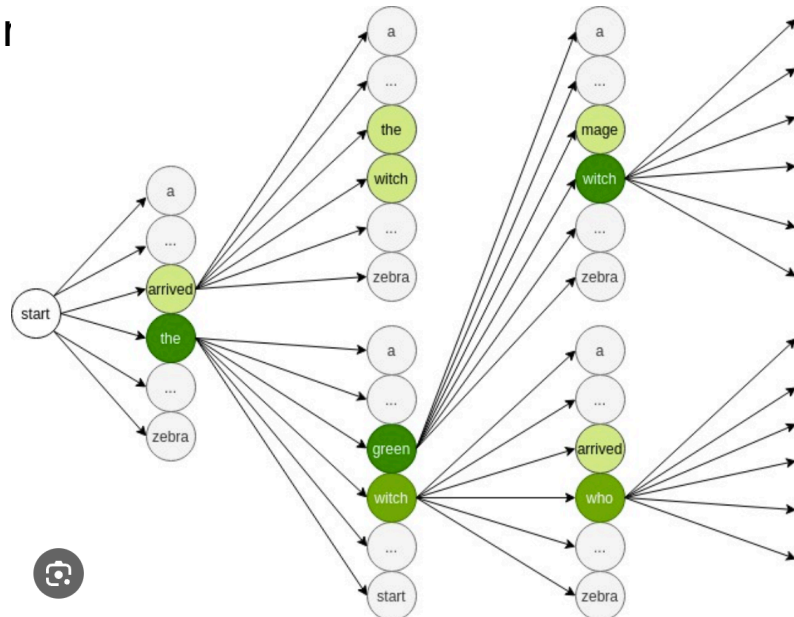
Pros: better likelihood solutions

$$O(t \cdot (B \cdot \text{compute} + B \cdot V \log(B)))$$

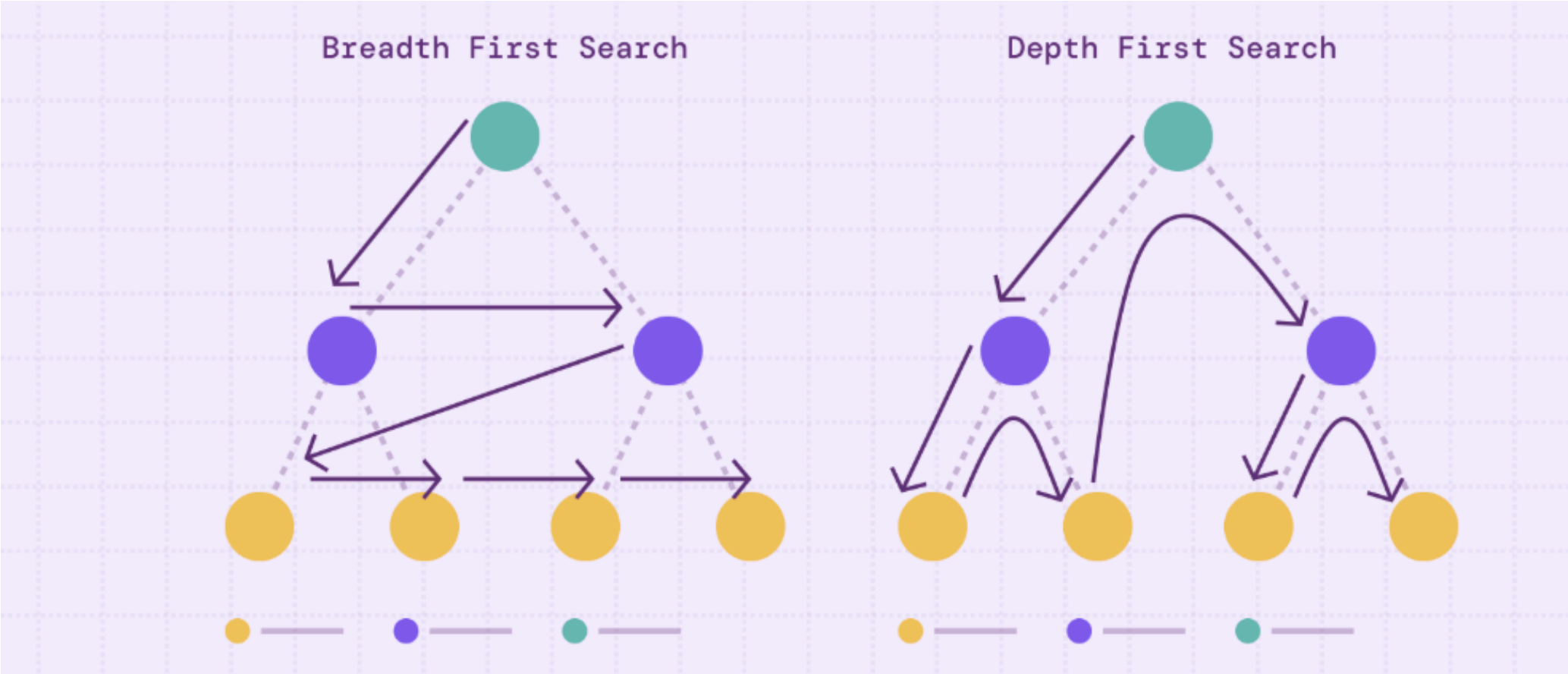
Cons:

-slower can produce **generic / repetitive** text; needs length penalty,

-Early probable sequences i
lower probs early on



BFS/DFS



Test-time compute scaling

Idea: trade **more inference compute** for **better quality** (accuracy, robustness, safety) *without retraining*.

1) Depth / Refinement (more thinking per answer)

Examples: Chain-of-Thought (CoT), self-reflection, draft→critique→revise

How it works

Generate an initial solution

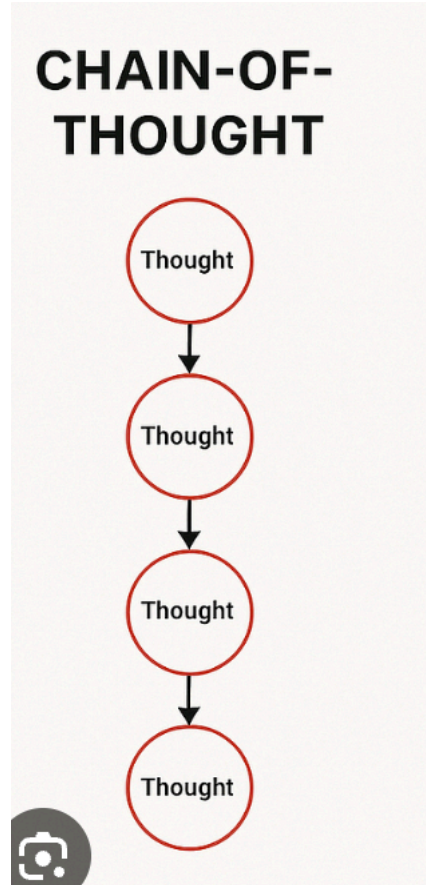
Spend extra passes to **check**, **fix**, and **tighten** reasoning

Optionally verify against constraints/tests

When it helps

Errors are from **reasoning mistakes** or missing steps

You can improve by iterative editing (math, logic, writing, planning)



Test-time compute scaling

2) Width / Repetition (more candidates, then select/aggregate)

Examples: Best-of-N, majority vote

How it works

Sample **N diverse candidates** (different beams, sampling, prompts)

Either:

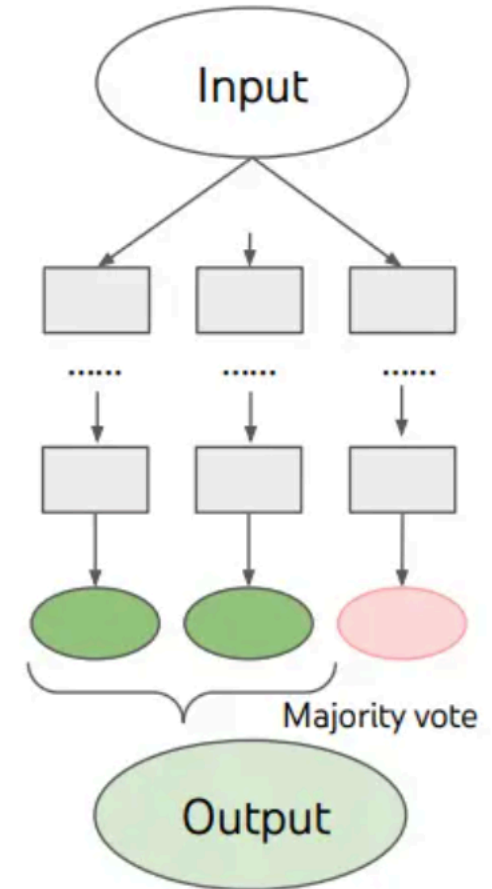
select best (reranker/verifier/judge), or

aggregate (vote/median/consensus)

When it helps

Model output has **high variance** (sometimes right, sometimes wrong)

You have a **verifier** (unit tests, schema checks, constraints, LLM-judge)



Test-time compute scaling

2) Width / Repetition (more candidates, then select/aggregate)

Examples: Best-of-N, majority vote

How it works

Sample **N diverse candidates** (different beams, sampling, prompts)

Either:

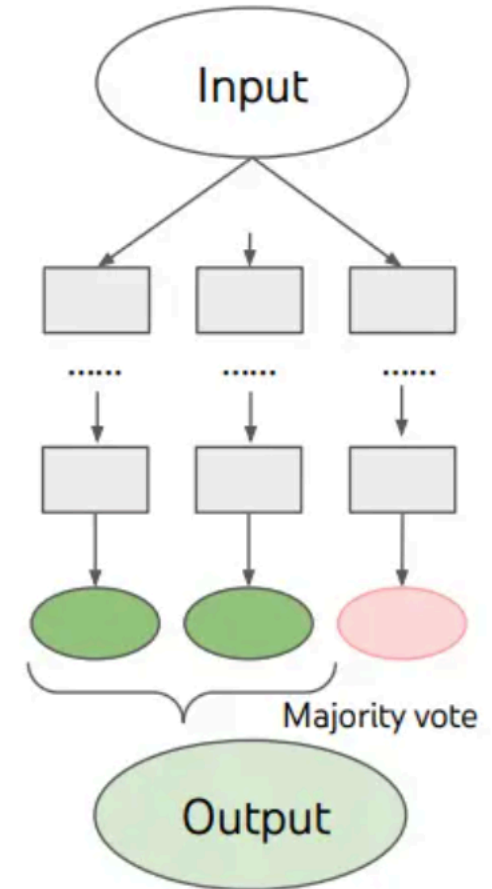
select best (reranker/verifier/judge), or

aggregate (vote/median/consensus)

When it helps

Model output has **high variance** (sometimes right, sometimes wrong)

You have a **verifier** (unit tests, schema checks, constraints, LLM-judge)



Test-time compute scaling

3) Combined / Tree Search (depth + width)

Examples: Tree-of-Thought (ToT), beam search, MCTS-style search

How it works

Explore multiple partial solutions (branches)

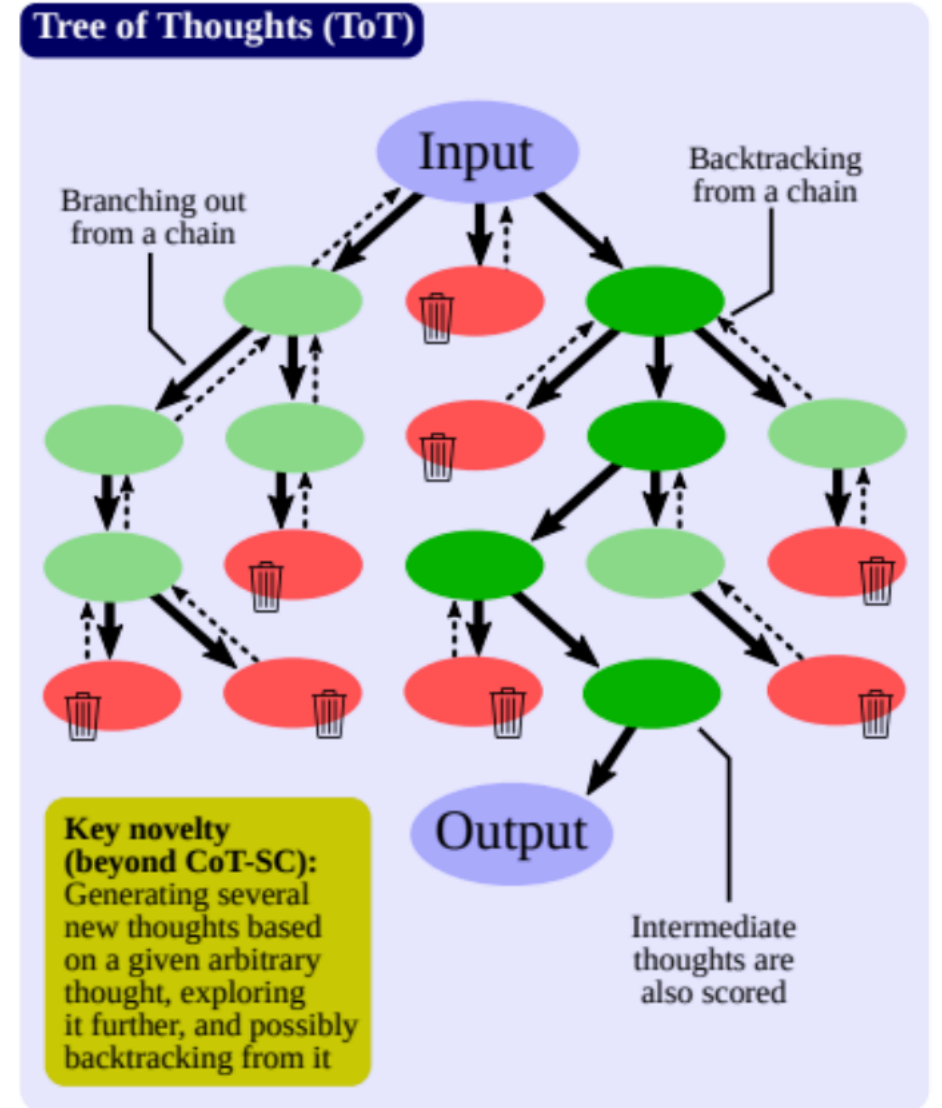
Use a **value function / verifier** to prune

Allocate compute to promising paths

When it helps

Multi-step problems: planning, reasoning with tools, complex constrain

You need **backtracking** and exploration



KV CACHE

KV cache

What it is: per-layer **Key/Value (K,V)** tensors for **all past tokens** stored during autoregressive decoding.

Why we need it: avoids recomputing attention K/V for the whole prefix every step.

No KV cache: per token you redo work for all previous tokens $O(t^2 \cdot L \cdot \text{head_dim})$ to recompute attention → slow (extra repeated compute).

With KV cache: compute K/V once per token $O(t \cdot L \cdot \text{head_dim})$, then reuse → much faster decoding.

Where stored: HBM (for low latency); can be **CPU/offloaded** if VRAM is tight (slower).

Problem Memory scaling: KV cache size grows $\sim O(\text{Batch} \cdot L_{\text{ayers}} \cdot \text{head_dim} \cdot \text{nbr_heads} \cdot t)$

.

Two methods: implicit (cache eviction) or explicit (prompt compression)

Implicit compression: KVzip: Query-Agnostic KV Cache Compression with Context Reconstruction

- Paper: <https://openreview.net/forum?id=JFygzwx8SJ>
- Many problems can be described as context+ set of questions
- objective: can we perform kv eviction on the context tokens to retain the most important info?

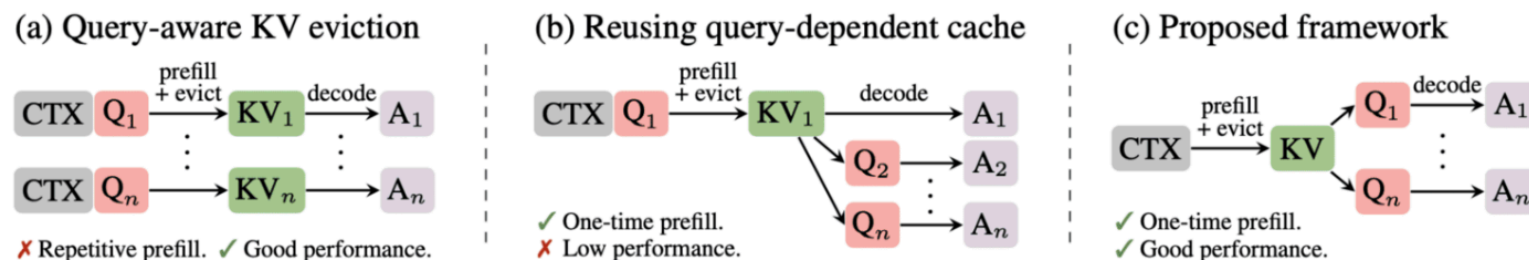


Figure: Overview of KV eviction strategies in multi-query scenarios. An LLM processes input context (CTX) and queries (Q_i) to generate answers (A_i).

idea: during prefill perform the maximal needed eviction that allows reconstruction

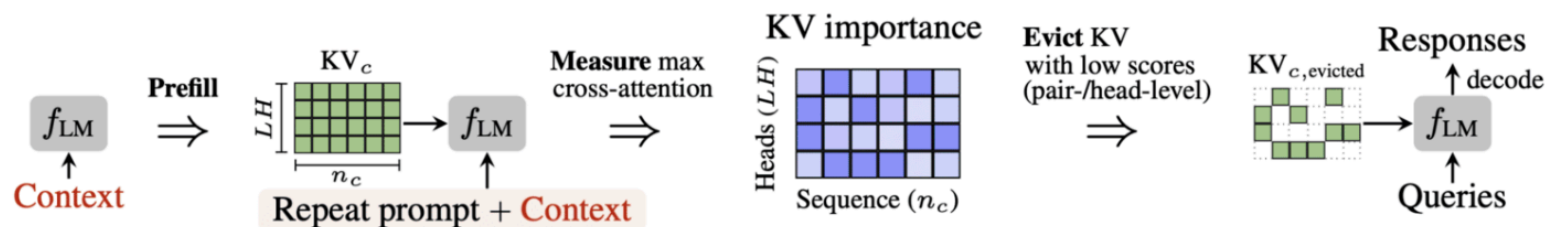


Figure: Method overview. KVzip evicts KV pairs with the lowest importance scores, accommodating both KV pair-level and head-level eviction.

Implicit compression: Streaming Attention Approximation via Discrepancy Theory

- Paper: <https://openreview.net/forum?id=p3HBEtNDY>

- Key ideas : 1) design a approximate version of attention which requires less memory
- Approach: design an algorithm that allows to find C' such that:

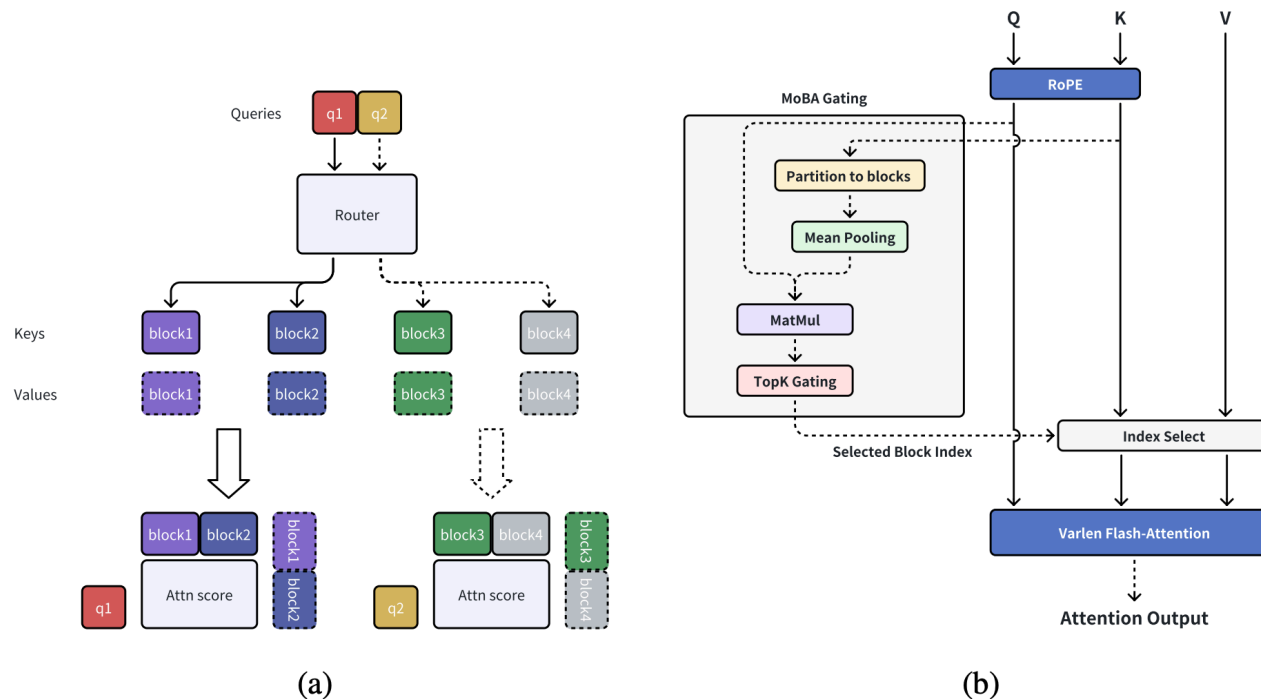
$$\sum_{\{k,v\} \in C'} \exp\left(\frac{\langle k, q_n \rangle}{\sqrt{d}}\right) v \approx \sum_{\{k,v\} \in C \setminus C'} \exp\left(\frac{\langle k, q_n \rangle}{\sqrt{d}}\right) v.$$

This allows to reduce kV cache to only the most important tokens C' , reducing memory requirement during inference

$$\sum_{\{k,v\} \in C} \exp\left(\frac{\langle k, q_n \rangle}{\sqrt{d}}\right) v \approx 2 \sum_{\{k,v\} \in C'} \exp\left(\frac{\langle k, q_n \rangle}{\sqrt{d}}\right) v.$$

Implicit compression: MoBA: Mixture of Block Attention for Long-Context LLMs

- Paper: <https://openreview.net/pdf?id=RIqYCpTu1P>
- Problem: how to overcome quadratic cost in very long context +1-10Mio
- Idea: inspired by the gated mechanism in MLP layers of MoE, they do the same for attention blocks during prefill

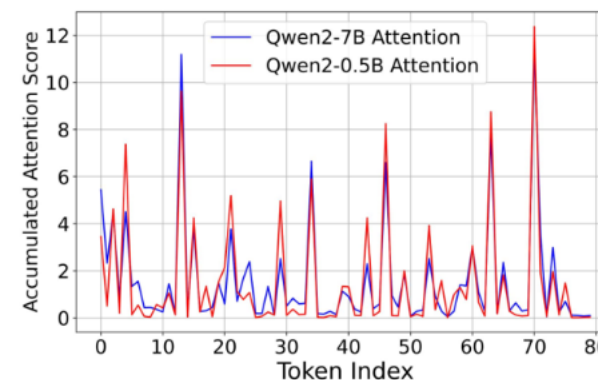


Implicit compression: SmallKV: Small Model Assisted Compensation of KV Cache Compression for Efficient LLM Inference

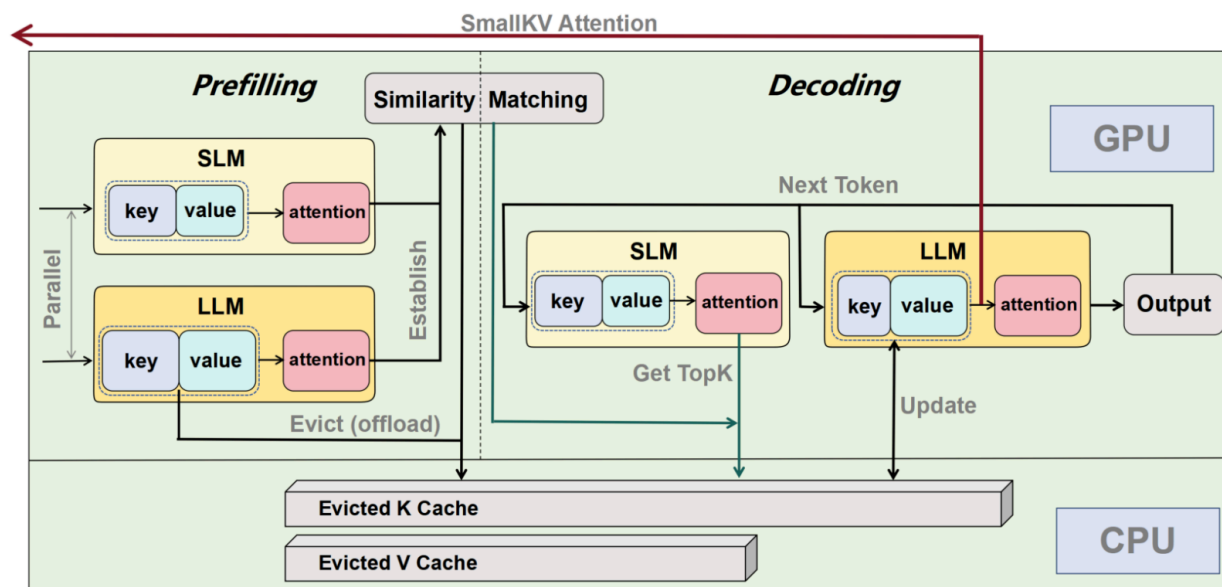
- Paper: <https://openreview.net/forum?id=0BVrpXMr5Y>

- Key insight: Llms of different size within the same series tend exhibit highly correlated attentions patterns

Their Idea: use a small language model (SLM) of the same series of you To identify most important KV and remove them smartly before processing with your LLM



(c) The visualization of highly consistent attention patterns between LLM and SLM.



My proposal: use a distilled model as your SLM

Deployment optimizations

Quantization

Quantization = represent weights/activations with fewer bits

Common inference targets:

int8 (often activations + weights)

int4 (usually **weight-only**)

Why it helps:

Memory ↓ (weights + sometimes KV-cache)

Bandwidth ↓ → often the real bottleneck

Potential **throughput** ↑ if kernels/hardware support is strong

Main tradeoff: **accuracy drop** vs compression / speed

Mixed Precision

Mixed Precision compute (bf16 / fp16)

Use lower precision floating point to accelerate matmuls on Tensor Cores

fp16

Faster on many GPUs, but smaller exponent → **overflow risk** in some ops

bf16

Same exponent range as fp32 → **more numerically stable**

Widely supported on newer hardware

->**faster**

Typical recipe:

Weights in **bf16/fp16**, compute in **tensor cores**, accumulate carefully

Keep sensitive ops in fp32 if needed (layernorm/softmax sometimes)

Puning & Sparsity (Structured vs Unstructured)

Unstructured pruning

Remove individual weights (magnitude $|w| < k$, gate-based pruning (Mask-gate + L1))

Pros: high sparsity possible; cons: speedup requires specialized sparse kernels → often limited real gains

Cons: can hurt performance a lot sometimes

Structured pruning

Remove entire units: heads, MLP neurons, blocks

Techniques: Mask-gate + L1 (learn which heads matter) ->

Pros: speed up can be great

Cons: quality might take a hit, requires a few finetuning steps
requires, carefully wiring to match dimension

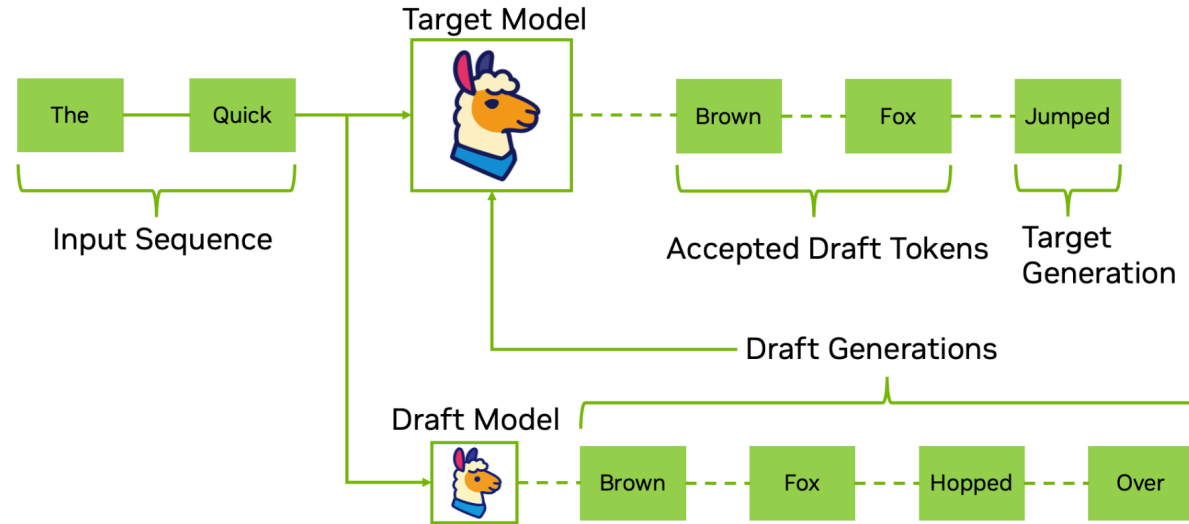
fine-tuning

- Usually requires some finetuning

ONNX → TensorRT

- (what it does):** compiles the exported graph into a GPU-specific engine with **op fusion**, **kernel/tactic auto-selection**, and **memory planning** (often FP16/INT8/INT4 where supported) → fewer kernel launches + less VRAM traffic.
- Use it when:** you have **stable shapes** (or can define good min/opt/max profiles), and your model is mostly **TensorRT-supported** (few custom/unsupported ops).
- Don't use it when:** your workload has **highly variable shapes**, many **unsupported ops** (causing fallbacks/graph splits)
- Reality check:** not guaranteed faster—**benchmark prefill vs decode**

Speculative decoding



Given a draft model (SLM) much cheaper than target model to run inference

AdaSPEC: Selective Knowledge Distillation for Efficient Speculative Decoders

- Paper: <https://openreview.net/forum?id=zNLlgISOwD>

- Key ideas : 1) current popular approaches for distillation (DistillSpec) optimize for KL not acceptance rate
- 2) acceptance rates can be significantly improved when focusing distillation on mastering easy tokens rather than wasting

Limited capacity on hard to learn tokens

Stage 1: Build a Reference Model

- Train M_{ref} from target M_p using standard KD (e.g., DistillSpec)
- M_{ref} shares architecture with draft model M_q

Stage 2: Select Learnable Tokens

- Compute per-token KL losses:

$$L_{\text{ref}}(w) = \text{KL}(P(w||c)||R(w||c)), \quad L_{\text{draft}}(w) = \text{KL}(P(w||c)||Q(w||c))$$

- Define token “learnability” by margin:

$$\Delta L(w) = L_{\text{draft}}(w) - \overset{\uparrow}{L}_{\text{ref}}(w)$$

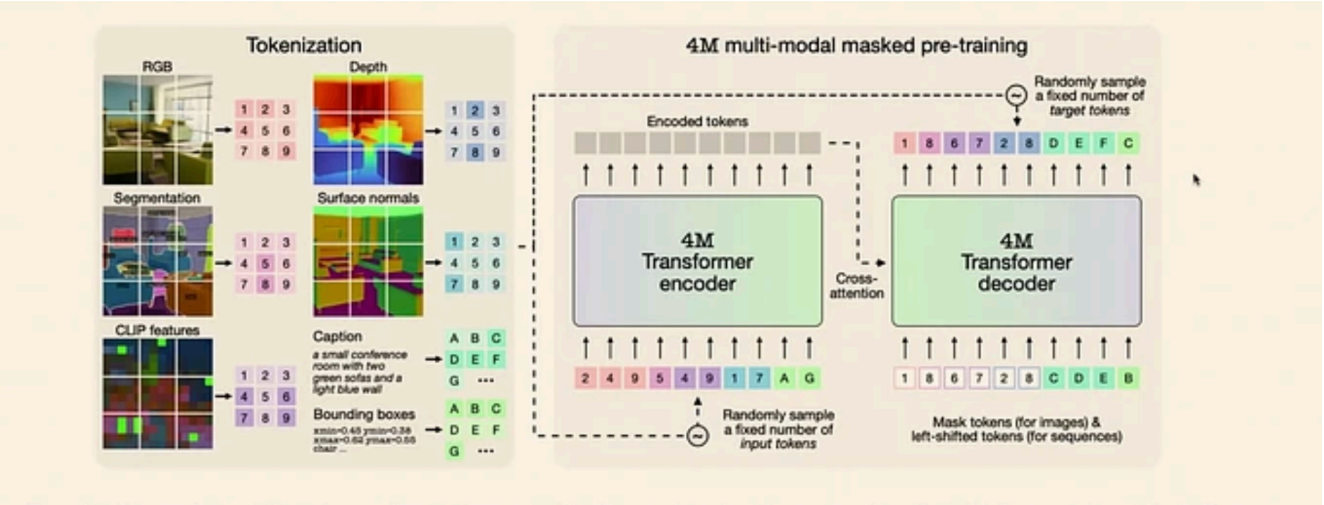
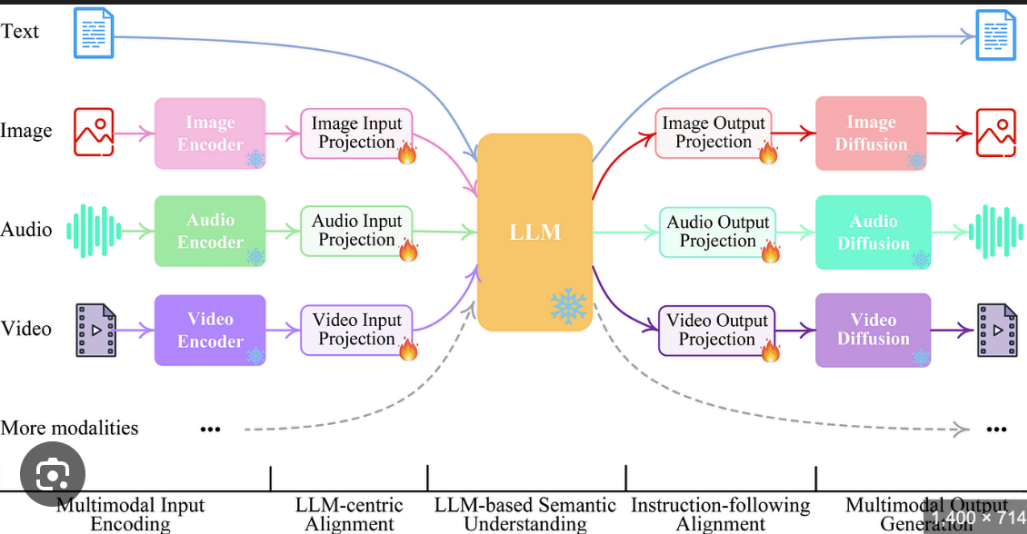
- Select top- k tokens with largest $\Delta L(w)$ (most improvable)

Quantization

- If bottleneck is **autoregressive loop latency** → **Speculative decoding**
- If bottleneck is **memory bandwidth / model size** → **Quantization (int4/int8)**
- If bottleneck is **compute / tensor core utilization** → **Mixed precision (bf16/fp16)**
- If you can afford training / architecture changes → **Structured/unstructured pruning** (+ finetune)
- If **model is too big** -> pipeline or tensor sharding

Multimodal LLMs

Multimodal architecture



$$\mathcal{L}_{\text{LM}} = - \sum_{t=1}^T \log p_{\theta}(y_t \mid y_{<t}, x_{\text{image}}, x_{\text{prompt}})$$

Vision transformer (ViT)

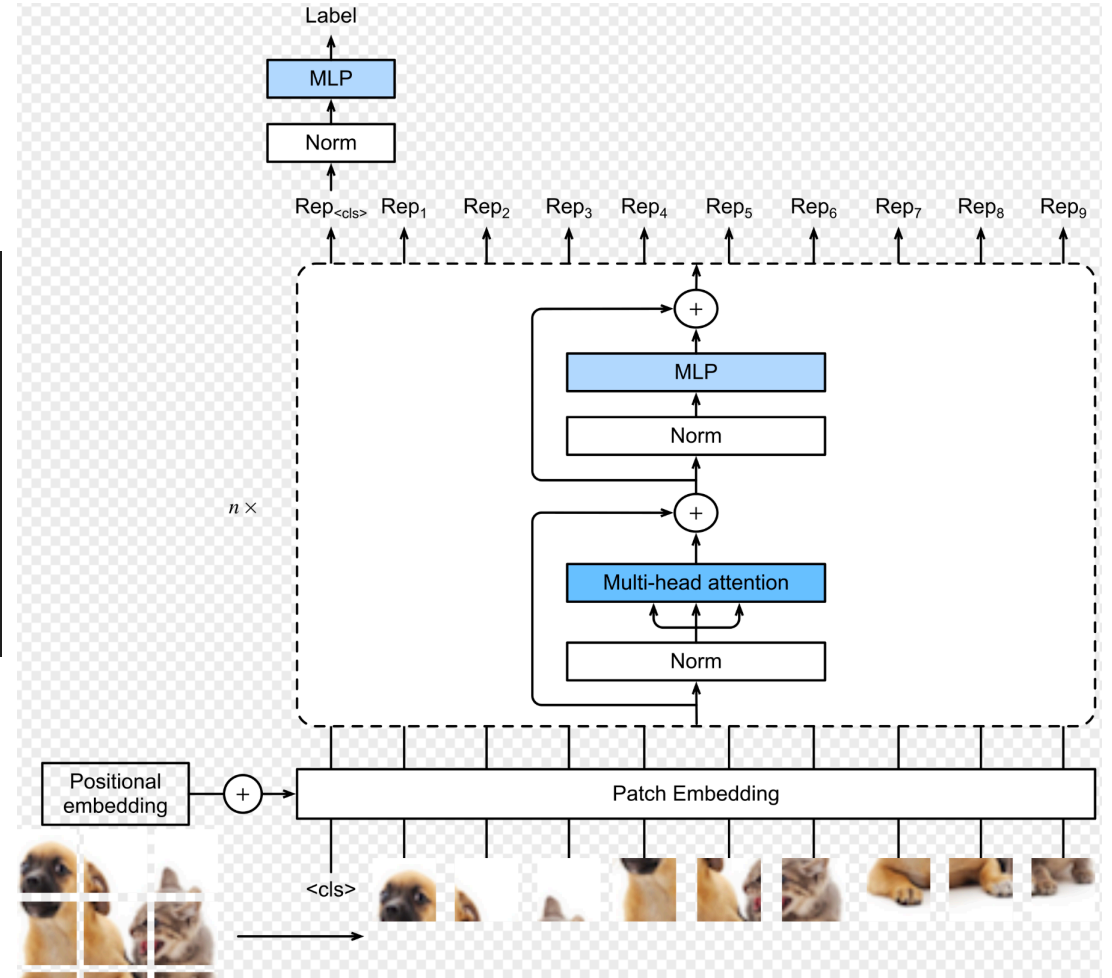
1. Resize/normalize image $\rightarrow$ tensor $(H, W, 3)$
2. Split into non-overlapping patches of size $P \times P$
Number of patches:

$$N = \frac{H}{P} \cdot \frac{W}{P}$$

3. Flatten each patch $(P \cdot P \cdot 3)$ and apply a linear layer to get an embedding of size D :

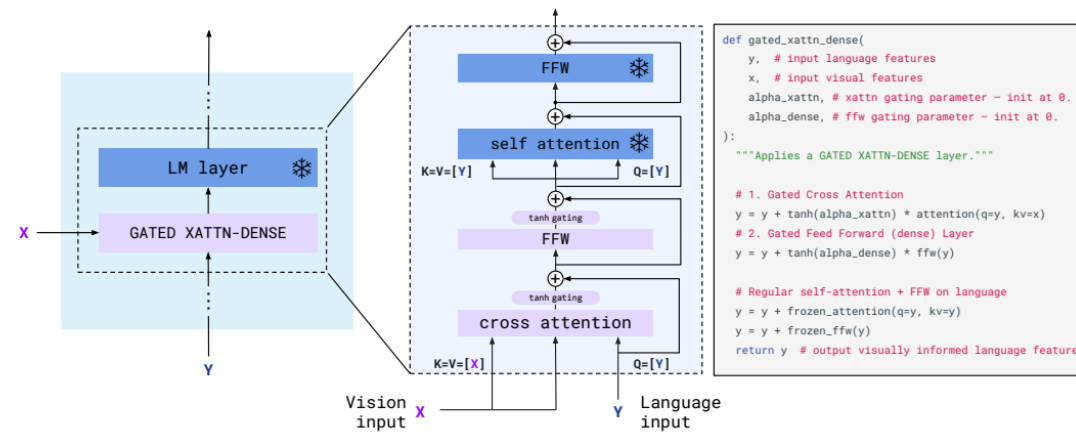
$$\text{patch_emb}_i = \text{Linear}(\text{flatten}(\text{patch}_i)) \in \mathbb{R}^D$$

4. The sequence $\{\text{patch_emb}_1, \dots, \text{patch_emb}_N\}$ are your "image tokens" (continuous vectors).

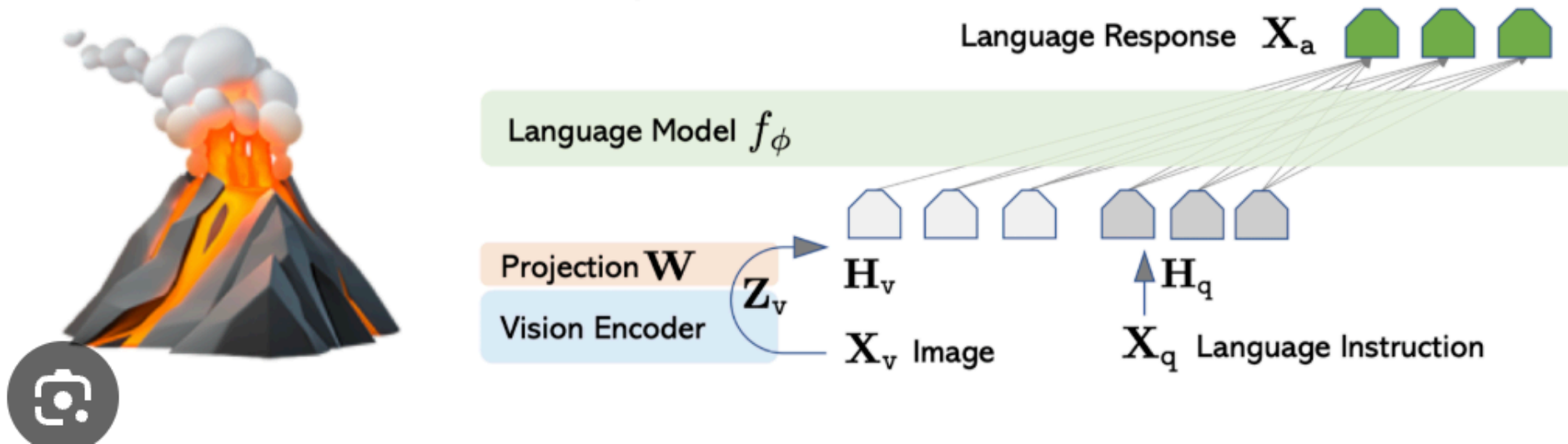


bert like classification:

Encoder + **cross-attention** into the LLM (Flamingo-like)



Adapter / projection into token space (LLaVA-like)



C) Unified tokenization (single autoregressive stream over modalities)

Here images/audio become **discrete token IDs** so the model predicts them like words.

Vector quantization (VQ) tokenization

- Encoder makes latents $z_i \in \mathbb{R}^d$
- Codebook $E = \{e_1, \dots, e_K\}$
- Discretize:

$$k_i = \arg \min_{k \in \{1..K\}} \|z_i - e_k\|^2$$

Now the image is a sequence of IDs $(k_1, \dots, k_M)$.

Unified autoregressive loss

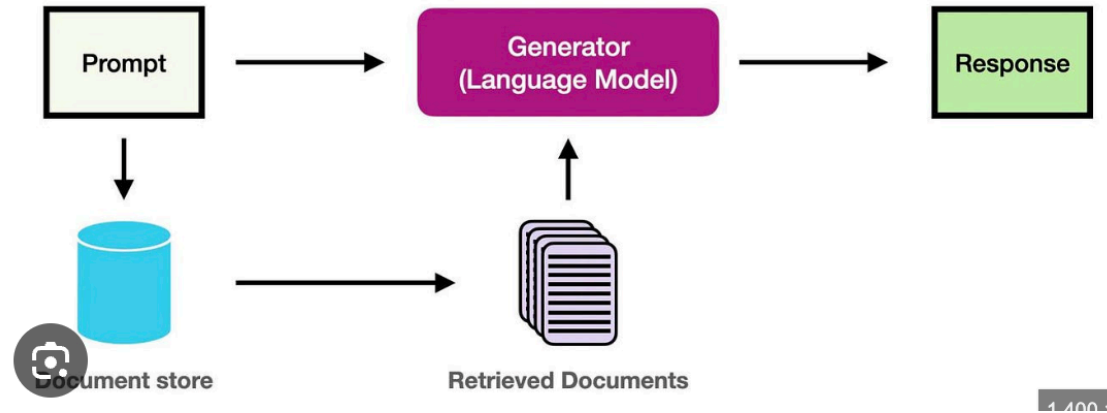
Concatenate text + special separators + image/audio tokens:

$$x = [\text{<txt>}, t_{1:n}, \text{}, k_{1:M}, \dots]$$

Train:

$$\mathcal{L} = - \sum_{t=1}^{|x|} \log p(x_t \mid x_{<t})$$

RAG (Retrieval-Augmented Generation)



What it is

A pipeline where the LLM **doesn't rely only on its weights**. Instead, it **retrieves relevant external text** (docs, tickets, webpages, DB rows) and uses it in the prompt to answer.

Why we do it

Freshness: knowledge can be updated without retraining the model

Accuracy / fewer hallucinations: answers are grounded in retrieved sources

Scalability: store *huge* corpora externally (millions of docs)

Auditability: can cite the chunks that supported the answer

RAG how

1) Indexing time (offline)

1. **Collect documents** (PDFs, web pages, wikis, tickets, code, etc.)
2. **Chunk** them into passages (e.g., 200–800 tokens with overlap)
3. For each chunk, compute an **embedding vector** $e = f_{\text{embed}}(\text{chunk})$
4. Store in a **vector index** (FAISS / ScaNN / Pinecone / etc.) with metadata (doc id, URL, timestamp, permissions)

Result: you can do fast **nearest-neighbor search** over chunk embeddings.

2) Query time (online)

Given user question q :

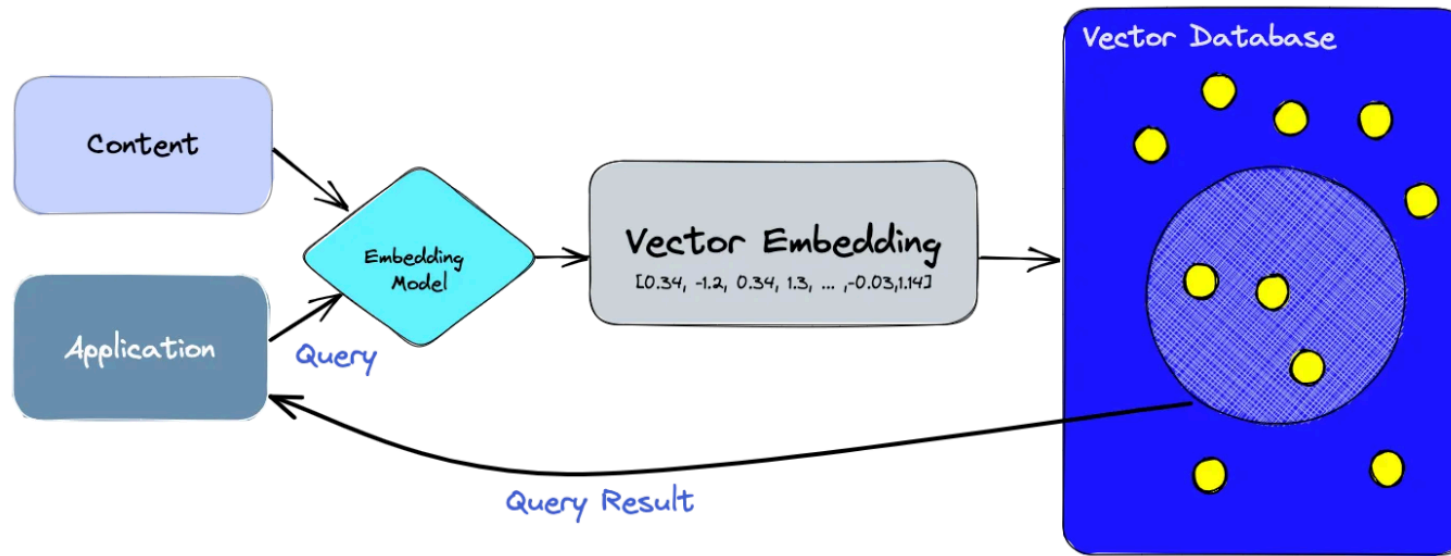
A) Retrieve

1. Embed the query: $v_q = f_{\text{embed}}(q)$
2. Find top- k chunks by similarity (cosine/dot product):

$$\text{topK} = \arg \max_i \text{sim}(v_q, e_i)$$

3. (Often) **filter** by metadata (access control, recency, language, source)
4. (Often) **rerank** retrieved chunks using a cross-encoder / LLM scorer for better precision

Faiss



Multiple ways

1: Flat (exact) search — “compare to everything”

Slow

2: “cluster first, search a few clusters”

BENCHMARK AND EVALUATION

Most-used LLM metrics Model quality

Loss / Perplexity (PPL): next-token prediction quality (pretraining, eval on held-out text)

$$\text{PPL} = \exp \left(-\frac{1}{N} \sum_{t=1}^N \log p(x_t \mid x_{<t}) \right)$$

-> PPL = 10 ≈ the model behaves like on average it has 1/10 prob to predict correct token

Task success: Accuracy / EM / F1 (QA), **Pass@1 / Pass@k** (code), benchmark scores (reasoning/knowledge)

Generation quality (open-ended)

Win-rate / preference rate: human or model-judge pairwise comparisons (most common for chat)

Alignment & safety

Violation rate / refusal accuracy: safety policy compliance, jailbreak success rate

Serving efficiency

Throughput: tokens/sec (and requests/sec)

Memory & cost: (weights + KV cache). \$/1M tokens